

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG



BÁO CÁO THỰC NGHIỆM

Học phần: An toàn & Bảo mật thông tin

**Chủ đề: Xây dựng chương trình mã hóa và giải mã
Elgamal (Sử dụng ngôn ngữ Python, C++)**

Giáo viên hướng dẫn : TS. Phạm Văn Hiệp

Nhóm sinh viên thực hiện :

- | | |
|---------------------|-------------------|
| 1. Phạm Văn Thế Anh | Mã SV: 2024602880 |
| 2. Nguyễn Đức Huy | Mã SV: 2024605697 |
| 3. Đỗ Phương Linh | Mã SV: 2024607016 |
| 4. Ngô Đức Mạnh | Mã SV: 2024606404 |
| 5. Phạm Trọng Tiến | Mã SV: 2024604207 |

Mã Lớp học phần: 20252IT6002014

Nhóm: 08

Hà Nội - Năm 2026

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đề tài, nhóm chúng em đã nhận được rất nhiều sự quan tâm, giúp đỡ và hỗ trợ quý báu từ nhiều phía. Đây chính là nguồn động lực to lớn để chúng em có thể hoàn thành bài báo cáo này một cách tốt nhất.

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc tới **TS. Phạm Văn Hiệp** – người đã tận tình hướng dẫn, định hướng và truyền đạt cho chúng em những kiến thức nền tảng cũng như kinh nghiệm thực tiễn vô cùng quý giá. Không chỉ dừng lại ở việc giảng dạy lý thuyết, thầy còn luôn theo sát, góp ý chi tiết và chỉnh sửa từng nội dung trong quá trình chúng em thực hiện đề tài. Sự nghiêm túc, tận tâm và trách nhiệm của thầy đã giúp chúng em có cái nhìn rõ ràng hơn về cách tiếp cận một chương trình mã hóa và giải mã theo hướng khoa học, logic và thực tiễn.

Bên cạnh đó, chúng em cũng xin gửi lời cảm ơn chân thành đến các thành viên trong nhóm – những người đã cùng nhau đồng hành, chia sẻ ý tưởng và nỗ lực không ngừng trong suốt quá trình làm việc. Mỗi cá nhân đều đóng góp một phần quan trọng, từ việc nghiên cứu tài liệu, phân tích yêu cầu, thiết kế hệ thống cho đến hoàn thiện báo cáo. Chính tinh thần trách nhiệm, sự chủ động và khả năng phối hợp hiệu quả đã giúp nhóm vượt qua những khó khăn, áp lực về thời gian cũng như khối lượng công việc để hoàn thành đề tài đúng tiến độ.

Thông qua quá trình thực hiện, chúng em không chỉ củng cố được kiến thức chuyên môn mà còn rèn luyện thêm nhiều kỹ năng quan trọng như làm việc nhóm, quản lý thời gian, tư duy hệ thống và giải quyết vấn đề. Đây sẽ là những hành trang quý báu cho quá trình học tập và công việc sau này.

Tuy nhiên, do kiến thức và kinh nghiệm thực tế còn hạn chế, bài báo cáo chắc chắn vẫn còn những thiếu sót nhất định. Nhóm chúng em rất mong nhận được sự góp ý, nhận xét từ thầy để có thể hoàn thiện hơn và nâng cao hiểu biết của mình trong tương lai.

Một lần nữa, chúng em xin chân thành cảm ơn!

LỜI MỞ ĐẦU

Trong bối cảnh cuộc cách mạng công nghiệp 4.0, sự phát triển mạnh mẽ của công nghệ thông tin và truyền thông đã kéo theo sự gia tăng nhanh chóng của các giao dịch điện tử và hoạt động lưu trữ dữ liệu số, vấn đề bảo mật thông tin ngày càng trở thành một yêu cầu vô cùng quan trọng. Đặc biệt, khi các thông tin quan trọng và dữ liệu nhạy cảm được trao đổi thông qua môi trường mạng, việc đảm bảo tính an toàn, bảo mật và toàn vẹn của dữ liệu là điều hết sức cần thiết. Một trong những giải pháp hiệu quả nhằm đáp ứng yêu cầu đó chính là việc ứng dụng các hệ mật mã hiện đại trong quá trình mã hóa và giải mã thông tin.

Trong số các hệ mật khóa công khai hiện nay, hệ mã ElGamal là một thuật toán mật mã phổ biến và được đánh giá cao nhờ tính bảo mật mạnh mẽ cùng khả năng triển khai hiệu quả trong nhiều hệ thống bảo mật thông tin. Hệ mã này được xây dựng dựa trên bài toán logarit rời rạc trong số học modulo – một bài toán có độ khó tính toán cao, qua đó giúp đảm bảo tính an toàn cho dữ liệu trong quá trình truyền nhận thông tin. Với cơ chế mã hóa và giải mã sử dụng cặp khóa công khai và khóa bí mật, ElGamal không chỉ hỗ trợ bảo vệ dữ liệu mà còn góp phần nâng cao tính bảo mật trong các ứng dụng thực tế.

Trong bài báo cáo này, nhóm em sẽ trình bày chi tiết quá trình nghiên cứu và xây dựng chương trình mã hóa và giải mã dựa trên hệ mã ElGamal. Bên cạnh đó, nhóm cũng sẽ tiến hành thử nghiệm chương trình, phân tích kết quả đạt được và đánh giá hiệu quả hoạt động của hệ mã trong việc bảo vệ thông tin. Thông qua đó, nhóm mong muốn làm rõ tính khả thi cũng như những ưu điểm nổi bật của hệ mã ElGamal trong lĩnh vực an toàn và bảo mật dữ liệu.

Qua quá trình nghiên cứu và thực hiện đề tài, nhóm em không chỉ nâng cao hiểu biết về các thuật toán mật mã hiện đại mà còn có cơ hội vận dụng kiến thức lý thuyết vào xây dựng một chương trình ứng dụng thực tế. Đồng thời, chúng em hy vọng rằng đề tài này sẽ góp phần hỗ trợ việc tìm hiểu và ứng dụng các giải pháp bảo mật thông tin trong môi trường số hiện nay.

MỤC LỤC

LỜI CẢM ƠN	2
LỜI MỞ ĐẦU	3
CHƯƠNG 1. TỔNG QUAN.....	8
1.1 Tổng quan về an toàn bảo mật thông tin.....	8
1.1.1 An toàn và bảo mật thông tin	8
1.1.2 Hệ mật mã.....	9
1.2 Các kiến thức cơ sở.....	10
1.2.1 Kiến thức toán học.....	10
1.2.2 Các thuật toán nền tảng trong cài đặt mật mã	12
1.2.3 Ngôn ngữ lập trình ứng dụng	13
1.3 Nội dung nghiên cứu.....	13
1.3.1 Lý do chọn đề tài	13
1.3.2 Nội dung nghiên cứu chính	14
1.3.3 Phương pháp nghiên cứu	14
CHƯƠNG 2. KẾT QUẢ NGHIÊN CỨU	16
2.1 Nghiên cứu, tìm hiểu hệ mã hóa khóa công khai.....	16
2.1.1 Tổng quan về mật mã khóa công khai.....	16
2.1.2 Các hệ mật mã khóa công khai tiêu biểu.....	17
2.1.3 Ứng dụng của mật mã khóa công khai	19
2.2 Nghiên cứu, tìm hiểu về hệ mật mã Elgamal	20
2.2.1 Cấu trúc và nguyên lý hoạt động của thuật toán Elgamal	20
2.2.2 Nguyên lý hoạt động.....	20

2.2.3	Ưu điểm và hạn chế của thuật toán Elgamal so với các phương pháp khác	22
2.2.4	Các ứng dụng thực tế của thuật toán Elmagal trong bảo mật dữ liệu.....	24
2.3	Thiết kế chương trình, cài đặt thuật toán	26
2.3.1	Kịch bản chương trình.....	26
2.3.2	Giới thiệu ngôn ngữ lập trình sử dụng để cài đặt thuật toán	28
2.3.3	Cài đặt thuật toán, giao diện chương trình	32
CHƯƠNG 3. KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM.....		51
3.1	Kiến thức và kỹ năng đã học được.....	51
3.1.1	Kiến thức chuyên môn.....	51
3.1.2	Kỹ năng thực hành và lập trình	51
3.1.3	Kỹ năng mềm và bài học kinh nghiệm.....	52
3.2	Bài học kinh nghiệm	52
3.3	Đề xuất về tính khả thi của chủ đề nghiên cứu, những thuận lợi, khó khăn	53
TÀI LIỆU THAM KHẢO		55

DANH MỤC HÌNH ẢNH

Hình 1.1. Mô hình cơ bản của truyền tin bảo mật	10
Hình 2.1. Sơ đồ giai đoạn mã hóa.....	21
Hình 2.2. Sơ đồ giai đoạn giải mã.....	21

DANH MỤC BẢNG

Bảng 2.1. So sánh thuật toán Elgamal và một số thuật toán khác	24
--	----

CHƯƠNG 1. TỔNG QUAN

1.1 Tổng quan về an toàn bảo mật thông tin

1.1.1 An toàn và bảo mật thông tin

An toàn bảo mật thông tin ngày càng trở thành một vấn đề quan trọng trong thời đại số, khi dữ liệu ngày càng trở thành tài sản quý giá của các tổ chức, doanh nghiệp và cá nhân. Việc bảo vệ thông tin khỏi các mối đe dọa từ môi trường mạng là cần thiết để duy trì sự tin cậy, bảo mật và tính toàn vẹn của dữ liệu. Những sự cố mất mát hoặc rò rỉ dữ liệu có thể gây tổn hại nghiêm trọng về tài chính, uy tín và tính bảo mật của tổ chức.

An toàn bảo mật thông tin là một lĩnh vực quan trọng trong công nghệ thông tin, nhằm bảo vệ dữ liệu khỏi các mối đe dọa như truy cập trái phép, thay đổi hoặc phá hủy dữ liệu. Các biện pháp bảo mật thông tin bao gồm mã hóa, xác thực, kiểm soát truy cập và giám sát hệ thống.

Mục tiêu của an toàn và bảo mật thông tin là đảm bảo duy trì được tính bí mật, tính toàn vẹn, tính sẵn sàng, tính xác thực và tính chống chối bỏ của dữ liệu:

- Tính bí mật: Là sự ngăn ngừa việc tiết lộ trái phép những thông tin quan trọng, nhạy cảm. Gồm 2 nội dung là Bí mật về dữ liệu và Quyền riêng tư.
- Tính toàn vẹn: Là sự phát hiện và ngăn ngừa việc sửa đổi trái phép về dữ liệu, thông tin và hệ thống. Do đó bảo đảm sự chính xác về dữ liệu và hệ thống. Gồm có toàn vẹn về dữ liệu và toàn vẹn của hệ thống.
- Tính sẵn sàng: Đảm bảo truy cập và sử dụng thông tin kịp thời và đáng tin cậy. Mất tính sẵn sàng là sự gián đoạn truy cập hoặc gián đoạn sử dụng thông tin hay hệ thống thông tin.
- Tính xác thực: Thể hiện thuộc tính được xác minh và có độ tin cậy; độ tin cậy vào tính hợp lệ của việc truyền thông, tin nhắn hoặc người khởi tạo tin nhắn.

- Tính chống chối bỏ: Mục tiêu an ninh quy định các hành động của một thực thể phải được quy về một cách duy nhất về thực thể đó. Điều này hỗ trợ chống từ chối, ngăn chặn và cách ly lỗi, phát hiện và ngăn chặn xâm nhập, phục hồi sau hành động và hành động pháp lý.

1.1.2 Hệ mật mã

1.1.2.1 Khái niệm cơ bản

Hệ mật mã (Cryptographic System) là một tập hợp các phương pháp, thuật toán, và giao thức được sử dụng để bảo vệ thông tin, đảm bảo tính bảo mật, toàn vẹn, và xác thực của dữ liệu trong quá trình truyền tải hoặc lưu trữ. Hệ mật mã giúp ngăn chặn việc truy cập trái phép, chỉnh sửa, hoặc đánh cắp thông tin thông qua việc chuyển đổi dữ liệu từ bản rõ (plain text) sang bản mã (cipher text) và ngược lại. Nói một cách đơn giản, đây là cách chúng ta "mã hóa" thông tin để chỉ những người có "chìa khóa" thích hợp mới có thể đọc được.

Trong thời đại số, thông tin là tài sản vô cùng quý giá. Hệ mật mã giúp chúng ta:

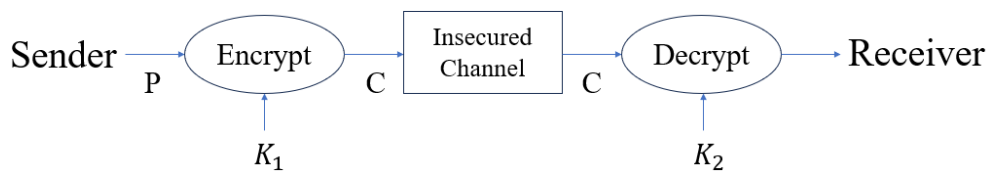
- Che giấu được nội dung của văn bản rõ (plain text)
- Tạo các yếu tố xác thực thông tin, đảm bảo thông tin lưu hành trong hệ thống đến người nhận hợp pháp là xác thực (authenticity)
- Tổ chức các sơ đồ chữ ký điện tử, đảm bảo không có hiện tượng giả mạo, mạo danh để gửi thông tin trên mạng

Có nhiều cách để phân loại hệ mật mã. Dựa vào cách truyền khóa có thể phân thành hai loại:

- Hệ mật mã đối xứng (mã hóa khóa bí mật): sử dụng một khóa duy nhất cho cả quá trình mã hóa và giải mã. Tốc độ xử lý mã hóa và giải mã dữ liệu nhanh, hiệu quả tuy nhiên vì khóa ở cả hai quá trình là như nhau nên cần chia sẻ khóa một cách an toàn, dễ bị lộ khóa nếu không quản lý tốt. Một số hệ mật đối xứng phổ biến như: DES (Data Encryption Standard), 3DES (Triple DES), AES (Advanced Encryption Standard)...

- Hệ mật mã bất đối xứng (mã hóa khóa công khai): sử dụng một cặp khóa: khóa công khai, khóa bí mật. Khóa công khai được chia sẻ rộng rãi, dùng để mã hóa dữ liệu. Khóa bí mật được giữ bí mật, dùng để giải mã dữ liệu. Vì khóa ở hai quá trình là riêng biệt nên đảm bảo an toàn hơn, dễ quản lý khóa. Tuy nhiên tốc độ xử lý mã hóa và giải mã dữ liệu chậm hơn so với hệ mật đối xứng. Một số hệ mật bất đối xứng phổ biến như: RSA (Rivest–Shamir–Adleman), ElGamal...

1.1.2.2 Các thành phần của hệ mật mã



Hình 1.1. Mô hình cơ bản của truyền tin bảo mật

Một hệ mật mã là bộ 5 (P, C, K, E, D) thỏa mãn các điều kiện sau:

- P (Plain text): không gian bản rõ, là tập hợp hữu hạn các bản rõ có thể có.
- C (Cipher text): không gian bản mã, là tập hợp những bản mã có thể có.
- K (Key): không gian khóa, là tập hợp hữu hạn các khóa có thể có.
- E (Encryption): không gian hàm mã hóa, là tập hợp các quy tắc mã hóa có thể có.
- D (Decryption): không gian hàm giải mã, là tập các quy tắc giải mã có thể có.

Đối với mỗi k thuộc K có một quy tắc mã hoá $e_k: P \rightarrow C$ thuộc E và một quy tắc giải mã tương ứng $d_k: C \rightarrow P$ thuộc D . Trong đó hàm giải mã d_k chính là ánh xạ ngược của hàm mã hóa e_k .

1.2 Các kiến thức cơ sở

1.2.1 Kiến thức toán học

Hệ mật mã khóa công khai nói chung và Elgamal nói riêng không dựa trên sự bí mật của thuật toán, mà dựa trên độ khó mang tính toán học của các

bài toán số học. Dưới đây là các kiến thức toán học nền tảng bắt buộc để cấu thành hệ mật Elgamal.

1.2.1.1 Phép toán số dư

Cho hai số nguyên a và n ($n > 0$). Phép toán $a \pmod n$ trả về số dư r của phép chia số nguyên a cho n , sao cho:

$$a = q \cdot n + r \text{ với } 0 \leq r < n$$

Các tính chất quan trọng được ứng dụng khi lập trình tính toán số lớn:

$$(a + b) \pmod n = [(a \pmod n) + (b \pmod n)] \pmod n$$

$$(a \cdot b) \pmod n = [(a \pmod n) \cdot b \pmod n] \pmod n$$

$$(a^b) \pmod n = [(a \pmod n)^b] \pmod n$$

1.2.1.2 Số nguyên tố và Tập hợp số nguyên đồng dư Z_p^*

- Số nguyên tố (p): Là số nguyên lớn hơn 1 và chỉ có hai ước số là 1 và chính nó. Trong Elgamal, người ta sử dụng các số nguyên tố cực kỳ lớn (thường từ 1024 bit đến 2048 bit trở lên) để đảm bảo an toàn.
- Tập hợp Z_p^* : Là tập hợp các số nguyên từ 1 đến $p-1$ nguyên tố cùng nhau với p . Vì p là số nguyên tố nên:

$$Z_p^* = \{1, 2, 3, \dots, p-1\}$$

1.2.1.3 Phần tử nguyên thủy

- Một số nguyên g được gọi là phần tử nguyên thủy của số nguyên tố p nếu các lũy thừa của g modulo p có thể sinh ra tất cả các phần tử của tập hợp Z_p^* .
- Nói cách khác, tập hợp sau phải trùng khớp với Z_p^* :

$$\{g^1 \pmod p, g^2 \pmod p, \dots, g^{p-1} \pmod p\} = \{1, 2, 3, \dots, p-1\}$$

- Ý nghĩa: Khi chọn được g , bất kỳ một giá trị x ngẫu nhiên nào trong khoảng từ 1 đến $p-1$ cũng sẽ tạo ra một kết quả $g^x \pmod p$ duy nhất và phân tán đều, giúp tối ưu hóa tính ngẫu nhiên của khóa mã hóa.

1.2.1.4 Bài toán Logarit rời rạc

Đây là "bức tường bảo mật" của hệ mật Elgamal. Bài toán được phát biểu như sau:

- Bài toán thuận (Dễ): Cho biết số nguyên tố p , phần tử nguyên thủy g , và một số nguyên x . Việc tính toán giá trị $y = g^x \pmod{p}$ là cực kỳ dễ dàng và nhanh chóng (sử dụng thuật toán bình phương và nhân liên tiếp).
- Bài toán nghịch (Cực khó): Cho biết p , g , và y . Yêu cầu tìm lại số nguyên x sao cho:

$$g^x \equiv y \pmod{p} \text{ (hay } x = \log_g y \pmod{p})$$

Hiện nay, với các số p đủ lớn, chưa có một thuật toán máy tính nào có thể giải được bài toán nghịch này trong thời gian đa thức. Kẻ tấn công biết khóa công khai (p, g, y) nhưng không thể nào tìm ra khóa bí mật (x) .

1.2.2 Các thuật toán nền tảng trong cài đặt mật mã

Khi hiện thực hóa hệ mật mã Elgamal trên máy tính với các số nguyên lớn, việc sử dụng các vòng lặp tuyến tính thông thường sẽ gây ra hiện tượng nghẽn thuật toán hoặc tràn bộ nhớ. Do đó, chương trình cần áp dụng các thuật toán tối ưu sau để tính toán:

- Thuật toán lũy thừa nhị phân (Modular Exponentiation): Dùng để tính toán nhanh giá trị $a^b \pmod{n}$ với độ phức tạp thời gian tối ưu là $O(\log b)$. Thuật toán này chia nhỏ số mũ b thành dạng nhị phân, giúp máy tính tính toán các lũy thừa hàng trăm chữ số chỉ trong vài mili-giây.
- Thuật toán Euclid mở rộng (Extended Euclidean Algorithm): Được sử dụng để tìm nghịch đảo modulo (Modular Multiplicative Inverse) trong quá trình giải mã, phục vụ cho việc tính giá trị số học nhằm phục hồi thông điệp gốc mà không cần dùng phép chia số nguyên lớn.
- Đánh giá độ phức tạp thuật toán (Big O Notation): Khái niệm dùng để phân tích hiệu năng và thời gian thực thi của chương trình. Việc tối ưu các hàm xử lý về mức thời gian logarit là điều kiện bắt buộc để hệ thống mã hóa hoạt động ổn định trên môi trường máy tính thực tế.

1.2.3 Ngôn ngữ lập trình ứng dụng

Để đánh giá và so sánh hiệu năng, đề tài tiến hành xây dựng và cài đặt chương trình trên hai ngôn ngữ lập trình có những đặc tính kỹ thuật hỗ trợ cho nhau:

- Ngôn ngữ Python: Là ngôn ngữ lập trình bậc cao, mã nguồn mở, nổi bật với cơ chế tự động quản lý bộ nhớ và hỗ trợ tính toán số nguyên lớn với độ chính xác vô hạn (Arbitrary-precision integers). Python giúp việc hiện thực hóa các công thức toán học của ElGamal trở nên trực quan, ngắn gọn và giảm thiểu lỗi tràn số (overflow).
- Ngôn ngữ C++: Là ngôn ngữ lập trình hướng đối tượng có hiệu năng cực cao, tốc độ xử lý tối ưu nhờ khả năng biên dịch trực tiếp ra mã máy và can thiệp sâu vào hệ thống. Việc cài đặt ElGamal trên C++ giúp tối ưu hóa tốc độ thực thi của các hàm băm, hàm sinh số ngẫu nhiên lớn và phép toán số dư phức tạp, đồng thời rèn luyện tư duy quản lý cấu trúc dữ liệu lớn (BigInt).

1.3 Nội dung nghiên cứu

1.3.1 Lý do chọn đề tài

Trong thời đại công nghệ số hiện nay, việc trao đổi và lưu trữ thông tin trên môi trường mạng ngày càng phổ biến, kéo theo nhiều nguy cơ mất an toàn dữ liệu như đánh cắp thông tin, truy cập trái phép hay giả mạo dữ liệu. Vì vậy, vấn đề bảo mật thông tin đã trở thành yêu cầu quan trọng đối với cá nhân, doanh nghiệp và các tổ chức. Để bảo vệ dữ liệu trước các nguy cơ trên, các hệ mật mã hiện đại đóng vai trò vô cùng cần thiết.

Trong số các thuật toán mã hóa khóa công khai hiện nay, hệ mã ElGamal được đánh giá là một trong những thuật toán có độ an toàn cao nhờ dựa trên bài toán logarit rời rạc – bài toán có độ khó tính toán lớn trong toán học. Thuật toán này không chỉ được ứng dụng trong mã hóa dữ liệu mà còn được sử dụng trong chữ ký số và nhiều hệ thống bảo mật hiện đại.

Xuất phát từ ý nghĩa thực tiễn đó, nhóm lựa chọn đề tài “Xây dựng chương trình mã hóa và giải mã ElGamal” nhằm tìm hiểu sâu hơn về nguyên lý hoạt động của hệ mật khóa công khai, đồng thời ứng dụng kiến thức đã học để xây dựng chương trình mô phỏng thuật toán trong thực tế. Qua đề tài, nhóm mong muốn nâng cao kiến thức về an toàn bảo mật thông tin cũng như kỹ năng lập trình và triển khai thuật toán mật mã.

1.3.2 Nội dung nghiên cứu chính

Đề tài tập trung nghiên cứu hệ mật mã khóa công khai ElGamal và xây dựng chương trình mô phỏng quá trình mã hóa, giải mã dữ liệu bằng thuật toán này. Nội dung nghiên cứu bao gồm việc tìm hiểu tổng quan về mật mã học, vai trò của các hệ mã hóa trong bảo vệ thông tin và các kiến thức toán học nền tảng liên quan như số nguyên tố, phép toán modulo, lũy thừa modulo và bài toán logarit rời rạc.

Bên cạnh đó, đề tài nghiên cứu nguyên lý hoạt động của hệ mật ElGamal gồm các bước sinh khóa, mã hóa và giải mã dữ liệu. Nhóm tiến hành phân tích cơ chế tạo khóa công khai và khóa bí mật, cách chuyển đổi bản rõ thành bản mã cũng như cách khôi phục thông điệp ban đầu từ dữ liệu đã mã hóa.

Sau khi hoàn thành phần lý thuyết, nhóm tiến hành xây dựng chương trình mã hóa và giải mã ElGamal bằng ngôn ngữ lập trình Python và C++. Chương trình thực hiện các chức năng cơ bản như sinh khóa, mã hóa dữ liệu và giải mã thông điệp. Đồng thời, nhóm cũng thực hiện kiểm thử chương trình với nhiều bộ dữ liệu khác nhau nhằm đánh giá độ chính xác, hiệu quả và tính ổn định của thuật toán trong thực tế.

1.3.3 Phương pháp nghiên cứu

Để thực hiện đề tài, nhóm sử dụng kết hợp phương pháp nghiên cứu lý thuyết và thực nghiệm. Trước hết, nhóm tiến hành thu thập, tìm hiểu và phân tích các tài liệu liên quan đến an toàn bảo mật thông tin, mật mã học và hệ mã hóa ElGamal từ giáo trình, sách tham khảo và các nguồn tài liệu chuyên ngành.

Tiếp theo, nhóm áp dụng phương pháp phân tích và thiết kế hệ thống để xây dựng chương trình mô phỏng thuật toán ElGamal. Các thuật toán toán học phục vụ cho quá trình mã hóa và giải mã được nghiên cứu và hiện thực hóa bằng ngôn ngữ lập trình Python và C++.

Sau khi hoàn thiện chương trình, nhóm tiến hành kiểm thử với nhiều trường hợp dữ liệu khác nhau để đánh giá khả năng hoạt động của hệ thống. Kết quả kiểm thử được sử dụng để phân tích hiệu quả của thuật toán, đồng thời kiểm tra tính chính xác và độ ổn định của chương trình mã hóa và giải mã.

CHƯƠNG 2. KẾT QUẢ NGHIÊN CỨU

2.1 Nghiên cứu, tìm hiểu hệ mã hóa khóa công khai

2.1.1 Tổng quan về mật mã khóa công khai

2.1.1.1 Khái niệm và sự ra đời

Được Whitfield Diffie và Martin Hellman giới thiệu lần đầu vào năm 1976. Khái niệm này ra đời nhằm giải quyết bài toán nan giải nhất của hệ mật mã khóa bí mật truyền thống: Vấn đề phân phối khóa trên các kênh truyền thông không an toàn và để cung cấp cơ chế chữ ký số (điều mà mã hóa đối xứng không làm được).

2.1.1.2 Nguyên lý hoạt động

Cặp khóa bất đối xứng: Hệ thống không dùng chung một khóa mà tạo ra một cặp khóa (Key Pair) có liên hệ mật thiết với nhau về mặt toán học:

- Khóa công khai (Public Key): Được công bố rộng rãi cho bất kỳ ai. Dùng để mã hóa dữ liệu hoặc xác minh chữ ký.
- Khóa bí mật (Private Key): Được chủ sở hữu giữ bí mật tuyệt đối. Dùng để giải mã dữ liệu hoặc tạo chữ ký số.

Cơ chế: Nếu thông điệp được mã hóa bằng Public Key của người nhận, thì chỉ Private Key tương ứng của người đó mới có thể giải mã được. Không thể (hoặc cực kỳ khó) dùng Public Key để tìm ra Private Key.

2.1.1.3 Quá trình trao đổi khóa

Loại bỏ kênh bảo mật: Hai bên (ví dụ Alice và Bob) không cần phải gặp mặt trực tiếp hay dùng một kênh bí mật để thống nhất khóa trước khi giao tiếp.

Quy trình:

1. Alice và Bob tự tạo ra cặp khóa (Public, Private) của riêng mình.
2. Alice gửi Public Key của mình cho Bob qua môi trường mạng mở (và Bob cũng làm vậy).
3. Khi Bob muốn gửi tin cho Alice, Bob dùng Public Key của Alice để mã hóa.
4. Kẻ gian dù bắt được gói tin cũng không thể giải mã vì không có Private Key của Alice.

2.1.1.4 Cơ sở toán học

Dựa trên các Hàm một chiều có cửa sập (Trapdoor one-way functions): Dễ dàng tính toán theo một chiều (mã hóa), nhưng cực kỳ khó tính toán theo chiều ngược lại (giải mã) nếu không biết "cửa sập" (Khóa bí mật).

Các bài toán nền tảng: Bài toán phân tích ra thừa số nguyên tố (hệ RSA), bài toán logarit rời rạc (hệ ElGamal, ECC).

2.1.1.5 So sánh hệ mã khóa công khai và hệ mã hóa bí mật

Hệ mã hóa bí mật (Khóa đối xứng - Symmetric):

- Ưu điểm: Tốc độ mã hóa/giải mã cực nhanh; kích thước khóa nhỏ (128-256 bit); thuật toán đơn giản, phù hợp để mã hóa lượng dữ liệu lớn.
- Nhược điểm: Quản lý và phân phối khóa vô cùng khó khăn (nếu có n người dùng, cần quản lý $n(n-1)/2$ khóa); không hỗ trợ chống chối bỏ (chữ ký số).

Hệ mã hóa công khai (Khóa bất đối xứng - Asymmetric):

- Ưu điểm: Giải quyết triệt để bài toán phân phối khóa (môi trường n người dùng chỉ cần n cặp khóa); hỗ trợ tốt chữ ký số và tính chống chối bỏ.
- Nhược điểm: Tốc độ xử lý rất chậm (chậm hơn hàng nghìn lần so với mã hóa đối xứng); yêu cầu kích thước khóa rất lớn (2048 - 4096 bit với RSA) để đảm bảo an toàn; tiêu tốn nhiều tài nguyên tính toán.

2.1.2 Các hệ mật mã khóa công khai tiêu biểu

2.1.2.1 Hệ mật RSA

Lịch sử và ý tưởng: Do Rivest, Shamir và Adleman phát minh (1977). Ý tưởng dựa trên tính toán số học module.

Thuật toán:

- Tạo khóa: Chọn 2 số nguyên tố lớn p, q . Tính $n = p \times q$ và hàm Euler $\varphi(n) = (p-1)(q-1)$. Chọn e sao cho $\text{UCLN}(e, \varphi(n)) = 1$. Tính d sao cho $e \times d \equiv 1 \pmod{\varphi(n)}$. Khóa công khai là (e, n) , khóa bí mật là (d, n) .
- Mã hóa: Bản rõ M , bản mã $C = M^e \pmod{n}$.
- Giải mã: $M = C^d \pmod{n}$.

Vấn đề an toàn: Độ an toàn dựa trên khó khăn của bài toán phân tích số nguyên lớn n ra 2 thừa số nguyên tố p và q . Có rủi ro về "điểm bất động" (bản mã trùng bản rõ).

2.1.2.2 Hệ mật ElGamal

Cơ sở toán học: Bài toán tính Logarit rời rạc trên trường hữu hạn Z_p .

Thuật toán:

- Tạo khóa công khai bằng hàm mũ module: $y = \alpha^a \pmod{p}$.
- Mã hóa bằng cách sinh thêm một số ngẫu nhiên k . Bản mã là một cặp số $C = (C_1, C_2)$.

Phân tích: Bản mã sẽ dài gấp đôi bản rõ. Việc chọn số ngẫu nhiên k rất quan trọng để đảm bảo mỗi lần mã hóa cùng một bản rõ sẽ ra bản mã khác nhau.

2.1.2.3 Hệ mật trên đường cong Elliptic (ECC)

Khái niệm: Sử dụng cấu trúc đại số của các đường cong elliptic trên các trường hữu hạn (phương trình dạng $y^2 = x^3 + ax + b$). Phép nhân module được thay thế bằng phép "cộng điểm" trên đường cong.

Độ an toàn và ưu thế: ECC cung cấp mức độ an toàn tương đương với RSA nhưng sử dụng kích thước khóa nhỏ hơn rất nhiều (Ví dụ: Khóa ECC 256-bit an toàn tương đương RSA 3072-bit). Rất phù hợp cho các thiết bị di động, thẻ thông minh có tài nguyên hạn chế.

2.1.2.4 Hệ mật Rabin

Thuật toán: Tương tự RSA nhưng sử dụng số mũ mã hóa cố định $e = 2$. Mã hóa đơn thuần là một phép bình phương module n . Giải mã là phép tính căn bậc 2.

Tính hiệu quả: Việc mã hóa diễn ra rất nhanh, nhưng giải mã phức tạp vì sẽ trả về 4 kết quả (4 nghiệm), yêu cầu người nhận phải có cơ chế nhận diện đâu là bản rõ ban đầu.

2.1.2.5 Một số hệ mật khóa công khai khác

Merkle - Hellman (Xếp ba lô): Dựa trên bài toán NP-khó là xếp ba lô. Rất nhanh nhưng hiện nay hầu hết đã bị chứng minh là không an toàn.

Chor - Rivest (CR): Cũng dựa trên bài toán xếp ba lô nhưng an toàn hơn Merkle-Hellman (đòi hỏi toán học rất phức tạp).

McEliece: Dựa trên bài toán giải mã các mã tuyến tính tổng quát (mã sửa sai). Có ưu điểm nổi bật là kháng lại được máy tính lượng tử, dù kích thước khóa công khai rất lớn.

2.1.3 Ứng dụng của mật mã khóa công khai

2.1.3.1 Quản lý và phân phối

Trong thực tế, người ta hiếm khi dùng mã khóa công khai để mã hóa dữ liệu lớn vì quá chậm. Thay vào đó, nó được dùng để "gói" và phân phối một Khóa phiên đối xứng (Symmetric Session Key).

Sau khi hai bên trao đổi được khóa phiên một cách an toàn bằng RSA/ECC, họ sẽ dùng khóa phiên đó để mã hóa khối lượng dữ liệu khổng lồ bằng thuật toán đối xứng (như AES) nhằm tối ưu tốc độ.

2.1.3.2 Chữ ký số (Digital Signature) và Xác thực

Khái niệm và vai trò: Người gửi băm văn bản (Hash), sau đó dùng Khóa bí mật của mình để mã hóa chuỗi Hash tạo thành Chữ ký số. Nó đảm bảo tính toàn vẹn của dữ liệu (không bị sửa đổi) và tính chống chối bỏ (người gửi không thể phủ nhận việc mình đã gửi văn bản).

Các lược đồ thông dụng: RSA Signature, Lược đồ ElGamal, DSS (Digital Signature Standard do NIST công bố), và ECDSA (trên đường cong elliptic).

2.1.3.3 Chứng chỉ số và hạ tầng khóa công khai

Sự cần thiết: Làm sao để biết Public Key bạn nhận được thực sự là của hệ thống ngân hàng chứ không phải của một hacker đang giả mạo?

Chứng chỉ số & CA: Tổ chức chứng thực (CA - Certificate Authority) đóng vai trò trung gian thứ 3 đáng tin cậy. CA sẽ kiểm tra danh tính của ngân

hàng và phát hành một "Chứng chỉ số" (chuẩn X.509) gắn kết danh tính của ngân hàng với Public Key của họ, và CA sẽ ký số lên chứng chỉ đó.

Hoạt động trong thực tế: Kiến trúc PKI là nền tảng cốt lõi bảo vệ Internet hiện đại: bảo mật trang web (HTTPS thông qua SSL/TLS), email bảo mật (PGP, S/MIME) và mạng riêng ảo (VPN).

2.2 Nghiên cứu, tìm hiểu về hệ mật mã Elgamal

Thuật toán Elgamal là một thuật toán mã hóa bất đối xứng được phát triển bởi Tahar Elgamal và năm 1985. Đây là một trong những thuật toán quan trọng trong lĩnh vực mật mã học hiện đại.

2.2.1 Cấu trúc và nguyên lý hoạt động của thuật toán Elgamal

Thuật toán Elgamal gồm 3 bước chính:

- Tạo khóa
- Mã hóa
- Giải mã

2.2.2 Nguyên lý hoạt động

Giai đoạn tạo khóa:

- Chọn số nguyên tố q đủ lớn
- + q cần đủ lớn để đảm bảo độ an toàn mật mã
- Chọn a là một căn nguyên thủy của q
- + $a \in q$
- + a và q là hai số nguyên tố cùng nhau, tức $\gcd(a, q) = 1$
- + $\text{Ord}_a(q) = \phi(a) = q - 1$ (tức a sinh ra toàn bộ phần tử trong Z_q^*)
- Chọn $X_A \in q - 1$
- + Đây là khóa riêng của người nhận
- Tính $Y_A = a^{X_A} \bmod q$
- + Đây là khóa công khai, dùng cho người gửi để mã hóa
- Kết quả xác định khóa:
- + Khóa công khai: $K_p = \{q, a, Y_A\}$

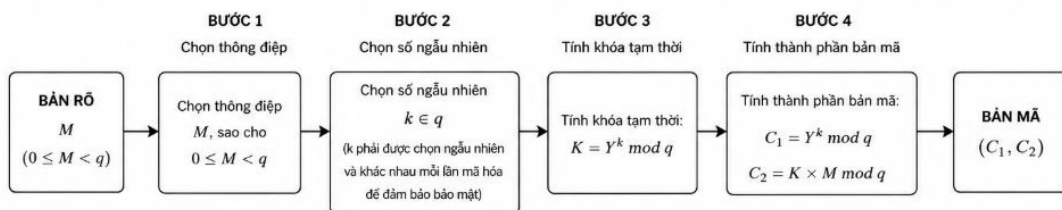
+ Khóa bí mật: $K_S = \{X_A\}$

Giai đoạn mã hóa:

- Chọn thông điệp ban đầu M , sao cho $0 \leq M < q$
- Chọn số ngẫu nhiên $k \in q$

k phải được chọn ngẫu nhiên và khác nhau mỗi lần mã hóa để đảm bảo bảo mật

- Tính khóa tạm thời: $K = Y^k \bmod q$
- Tính thành phần bản mã:
- + Thành phần thứ nhất: $C_1 = a^k \bmod q$
- + Thành phần thứ hai: $C_2 = K \times M \bmod q$
- Bản mã: $\{C_1, C_2\}$

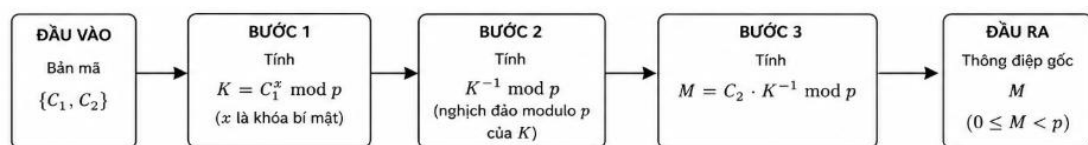


Hình 2.1. Sơ đồ giai đoạn mã hóa

Lưu ý: sau khi gửi bản mã đi thì số ngẫu nhiên k sẽ được hủy

Giai đoạn giải mã:

- Tính $K = C_1^X \bmod p$ (Vì $C_1^X \bmod p = a^{kX} \bmod p = Y^k \bmod p = K$)
- Tính khóa tạm thời: $K = C_1^{X_A} \bmod p$
- Tìm bản rõ: $M = C_2 \times K^{-1} \bmod p$



Hình 2.2. Sơ đồ giai đoạn giải mã

2.2.3 Ưu điểm và hạn chế của thuật toán Elgamal so với các phương pháp khác

Ưu điểm

- Bảo mật cao
- + Elgamal dựa vào bài toán logarit rời rạc trong trường hữu hạn, vốn là một bài toán “khó một chiều”. Để tính được X_A hoặc k , kẻ tấn công cần phải giải một trong 2 bài toán logarit rời rạc: $Y_A = a^{X_A} \bmod q$ hoặc $C_1 = a^k \bmod q$. Hiện chưa có thuật toán khả thi trong thời gian đa thức để giải bài toán n
- + Khi số nguyên tố q đủ lớn, thuật toán Elgamal không có phương pháp thám mã hiệu quả
- Tính ngẫu nhiên mạnh
- + Mỗi lần mã hóa cần một số ngẫu nhiên k , điều này giúp mỗi bản mã khác nhau, dù cho thông điệp và khóa là giống nhau.
- + Nhờ đó không thể đoán được nội dung bản mã từ một bản mã trước đó
- Không cần kênh an toàn để chia sẻ khóa bí mật
- + Chỉ có khóa công khai $\{q, a, Y_A\}$ được chia sẻ, khóa bí mật không bao giờ được truyền qua mạng
- Hỗ trợ chữ ký số
- + Elgamal có thể được mở rộng thành một hệ thống chữ ký số (Digital Signature Scheme)
- Mã nguồn mở, không bị cấp phép bản quyền
- + Thuật toán Elgamal không bị giới hạn bởi bản quyền, thường được sử dụng trong các phần mềm mã nguồn mở như GnuPG, PGP (phần mềm mã hóa dữ liệu và thư điện tử)

Hạn chế

- Hiệu suất thấp hơn RSA
- + Elgamal yêu cầu 2 phép mũ module lớn trong mỗi lần mã hóa và

giải mã:

- $C_1 = a^k \bmod q$
- + $K = Y_A^k \bmod q$
- + Trong khi đó, RSA chỉ cần một phép mũ module cho mã hóa/giải mã
- Elgamal chậm hơn đáng kể khi xử lý khối lượng lớn dữ liệu
- Kích thước bản mã lớn
- + Với mỗi thông điệp M , bản mã gồm 2 phần tử $\{C_1, C_2\}$, mỗi phần có độ dài tương đương với q
- + Tổng kích thước bản mã xấp xỉ gấp đôi độ dài bản rõ → tốn băng thông và dung lượng lưu trữ
- Sự phụ thuộc vào số ngẫu nhiên k
- + Số ngẫu nhiên k cần khác nhau mỗi lần mã hóa và không được để rò rỉ
- + Nếu sử dụng lại k , dễ bị tấn công như trong vụ tấn công chữ ký số Elgamal/DSA (điển hình là vụ hack Sony PlayStation 3 năm 2010).
- Tính bất thuận nghịch mã hóa
- + Khác với RSA, Elgamal không có tính đối xứng khóa công khai/khóa riêng:
 - RSA có thể sử dụng khóa riêng để mã hóa (ứng dụng trong chữ ký), còn Elgamal thì không
 - Điều này khiến Elgamal ít linh hoạt hơn trong 1 số giao thức mã hóa và xác thực

So sánh Elgamal và một số thuật toán khác:

Tiêu chí	Elgamal	RSA	DSA	ECC
Dựa trên	Logarit rời rạc	Phân tích số nguyên tố	Logarit rời rạc (dành cho chữ ký)	Logarit rời rạc trên đường cong elip
Dùng cho mã hóa	Có	Có	Không dùng để mã hóa	Có

Chữ ký số	Có	Có	Có	Có
Tốc độ mã hóa	Chậm hơn RSA	Nhanh hơn	Rất nhanh	Nhanh, tùy thuộc vào đường cong
Tốc độ giải mã	Chậm hơn RSA	Tương đối nhanh	Không áp dụng	Rất nhanh
Độ dài bản mã	Gấp đôi bản rõ	Bằng bản rõ	Không áp dụng	Nhỏ hơn Elgamal và RSA
Bảo mật	Rất cao nếu tham số an toàn	Dễ bị tấn công nếu khóa ngắn	An toàn nếu không bị lặp	An toàn với độ dài khóa ngắn
Kích thước khóa	Lớn hơn ECC/RSA để có mức an toàn tương đương	Trung bình	Trung bình	Nhỏ (256bit $\approx$ RSA 3072bit)

Bảng 2.1. So sánh thuật toán Elgamal và một số thuật toán khác

2.2.4 Các ứng dụng thực tế của thuật toán Elmagal trong bảo mật dữ liệu

Mã hóa dữ liệu (Data Encryption)

- ElGamal được dùng để mã hóa thông tin nhạy cảm trong quá trình truyền tải trên mạng (chẳng hạn như email, tập tin).
- Thực tiễn nhất là GnuPG (GPG): Một công cụ mã nguồn mở được sử dụng phổ biến trong Linux để mã hóa tập tin và thư điện tử. Elgamal được dùng để mã hóa key phiên (session key).

Chữ ký số (Digital Signature)

- ElGamal cũng được sử dụng trong việc tạo chữ ký số để xác thực người gửi và đảm bảo tính toàn vẹn của thông điệp.
- Thực tế, DSA (Digital Signature Algorithm) là một phiên bản chữ ký của Elgamal, được sử dụng trong nhiều hệ thống xác thực số như: SSH, SSL/TLS, X.509 certificate, chữ ký công cộng ở Việt Nam (VNPT-CA, FPT-CA...).

Xác thực và bảo mật email (Secure Email Authentication)

- Trong các hệ thống như PGP (Pretty Good Privacy) và GPG, Elgamal hỗ trợ mã hóa khóa phiên (session key) để gửi email an toàn, đảm bảo chỉ người nhận có khóa bí mật mới giải mã được nội dung.

Mã hóa trên nền tảng lưu trữ đám mây (Cloud Storage Encryption)

- Các hệ thống lưu trữ đám mây có thể sử dụng Elgamal để mã hóa dữ liệu người dùng trước khi lưu trữ nhằm tránh truy cập trái phép.
- Một số công cụ có thể tích hợp Elgamal để mã hóa khóa truy cập tệp tin trên đám mây như Google Drive, One Drive.
- Ngoài ra, Elgamal còn được sử dụng để mã hóa thông tin điều khiển (metadata/key).

Bảo mật trong giao tiếp mạng (Secure Communication)

- Elgamal có thể được tích hợp trong các giao thức bảo mật (SSL/TLS, IPsec...) nhằm đảm bảo thông tin trao đổi giữa các thiết bị là an toàn.
- IPSec và SSL/TLS trong một số cấu hình từng tích hợp các biến thể của Elgamal để mã hóa khóa trao đổi.
- Các hệ thống như VPN, HTTPS đôi khi dùng Elgamal để khởi tạo khóa.

Hệ thống bỏ phiếu điện tử (E-Voting Systems)

- Elgamal hỗ trợ các hệ thống bỏ phiếu mã hóa phiếu bầu một cách an toàn và ẩn danh, đồng thời vẫn có thể kiểm tra tính hợp lệ, đảm bảo tính riêng tư, tính toàn vẹn và không thể chối bỏ trong bầu cử điện tử.
- Helios Voting: Một hệ thống bỏ phiếu điện tử mã nguồn mở sử dụng Elgamal để đảm bảo tính riêng tư và kiểm chứng được.

Tiền mã hóa (Cryptocurrency) và Blockchain

- Một số giao thức blockchain nghiên cứu sử dụng biến thể của

Elgamal như Paillier, EC-Elgamal để xây dựng hệ thống giao dịch ẩn danh hoặc zero-knowledge proof.

- Trong các hệ thống như Confidential Transactions, Elgamal giúp mã hóa số tiền mà vẫn cho phép xác minh tính hợp lệ.

2.3 Thiết kế chương trình, cài đặt thuật toán

2.3.1 Kịch bản chương trình

Tên chương trình: Mô phỏng Mật mã Khóa công khai Elgamal

- Phân hệ 1: Giao diện chính (Menu)
 - + Mục tiêu: Điều hướng người dùng đến các chức năng của thuật toán Elgamal.
 - + Hiển thị: Giới thiệu ngắn gọn về thuật toán Elgamal dựa trên bài toán logarit rời rạc.
 - + Các lựa chọn (Nút bấm):
 1. Tạo khóa (Key Generation)
 2. Mã hóa thông điệp (Encryption)
 3. Giải mã thông điệp (Decryption)
- Phân hệ 2: Chức năng Tạo khóa (Key Generation)
 - + Mục tiêu: Sinh ra khóa công khai và khóa bí mật cho người dùng nhận (Ví dụ: Bob).
 - + Giao diện & Nhập liệu đầu vào:
 - Hệ thống yêu cầu nhập (hoặc nút bấm để hệ thống tự động sinh ngẫu nhiên) một số nguyên tố lớn p .
 - Yêu cầu nhập một phần tử sinh α của nhóm nhân Z_p^* .
 - Yêu cầu nhập số nguyên ngẫu nhiên a làm khóa bí mật, với điều kiện bắt buộc $1 < a < p - 2$.
 - Luồng xử lý (Logic code):
 - + Chương trình tính toán giá trị $\beta = \alpha^a \bmod p$.
 - + Đầu ra hiển thị:

- Khóa công khai (Public Key): Cung cấp công khai bộ ba số (p, α, β) để bất kỳ ai cũng có thể dùng gửi thông điệp.
- Khóa bí mật (Private Key): Giá trị α (hiển thị kèm cảnh báo phải giữ bí mật tuyệt đối).
- Phân hệ 3: Chức năng Mã hóa (Encryption)
 - + Mục tiêu: Người gửi (Ví dụ: Alice) sử dụng khóa công khai để mã hóa bản rõ gửi cho người nhận.
 - + Giao diện & Nhập liệu đầu vào:
 - Ô nhập Khóa công khai của người nhận: (p, α, β) .
 - Ô nhập thông báo gốc / Bản rõ x cần truyền đi (được chuyển đổi thành dạng số $x < p$).
 - Ô nhập hoặc sinh tự động một số nguyên ngẫu nhiên k (đóng vai trò là số bí mật dùng một lần của người gửi).
 - + Luồng xử lý (Logic code):
 - Bước 1: Tính thành phần thứ nhất của bản mã: $y_1 = x \cdot \alpha^k \bmod p$
 - Bước 2: Tính thành phần thứ hai của bản mã: $y_2 = x \cdot \beta^k \bmod p$
 - + Đầu ra hiển thị:
 - Bản mã (Ciphertext): Xuất ra cặp giá trị (y_1, y_2) để gửi qua kênh truyền thông không an toàn.
- Phân hệ 4: Chức năng Giải mã (Decryption)
 - + Mục tiêu: Người nhận dùng khóa bí mật của mình để khôi phục lại thông báo gốc.
 - + Giao diện & Nhập liệu đầu vào:
 - Ô nhập Bản mã: Cặp giá trị (y_1, y_2) nhận được từ đối tác.
 - Ô nhập Khóa bí mật cá nhân: a .
 - Ô nhập số nguyên tố modulo: p .
 - + Luồng xử lý (Logic code):
 - Bước 1: Tính toán khóa che giấu bằng công thức: $K = y_1^a \bmod p$.
 - Bước 2: Tìm nghịch đảo của K theo modulo p , ký hiệu là K^{-1} .

- Bước 3: Khôi phục bản rõ bằng công thức: $x = y_2 \cdot K^{-1} \bmod p$.
- + Đầu ra hiển thị:
- Bản rõ (Plaintext): Hiển thị nội dung thông báo x ban đầu đã được giải mã thành công.

2.3.2 Giới thiệu ngôn ngữ lập trình sử dụng để cài đặt thuật toán

2.3.2.1 Ngôn ngữ Python

- Tổng quan về Python: Python là một ngôn ngữ lập trình bậc cao, hiện đại được phát triển bởi Guido van Rossum. Python có cú pháp đơn giản, dễ đọc và dễ sử dụng, phù hợp với cả người mới học và lập trình viên chuyên nghiệp. Ngôn ngữ này hỗ trợ nhiều mô hình lập trình như lập trình hướng đối tượng, lập trình thủ tục và lập trình hàm. Python được ứng dụng rộng rãi trong nhiều lĩnh vực như phát triển web, trí tuệ nhân tạo, khoa học dữ liệu và bảo mật thông tin. Với hệ thống thư viện phong phú và khả năng xử lý linh hoạt, Python rất phù hợp để cài đặt các thuật toán mã hóa như ElGamal.
- Các đặc điểm nổi bật của Python
- + Hướng đối tượng: Python hỗ trợ đầy đủ các nguyên tắc của lập trình hướng đối tượng như tính kế thừa, đa hình và đóng gói. Điều này giúp chương trình được tổ chức rõ ràng, dễ quản lý và thuận tiện cho việc mở rộng hoặc nâng cấp sau này.
- + Cú pháp đơn giản, dễ đọc: Một trong những ưu điểm lớn nhất của Python là cú pháp ngắn gọn và gần gũi với ngôn ngữ tự nhiên. Điều này giúp người học dễ tiếp cận, đồng thời giúp lập trình viên giảm thời gian viết và kiểm tra mã nguồn.
- + Thư viện phong phú: Python sở hữu hệ sinh thái thư viện rất lớn, hỗ trợ mạnh mẽ cho nhiều lĩnh vực khác nhau. Đối với các bài toán mã hóa và xử lý thuật toán, Python cung cấp nhiều thư viện hỗ trợ tính toán số học, xử lý dữ liệu và thực hiện các phép toán phức tạp.

- + Khả năng đa nền tảng: Python có thể hoạt động trên nhiều hệ điều hành như Windows, Linux và macOS mà không cần thay đổi nhiều mã nguồn. Điều này giúp chương trình dễ dàng triển khai trên nhiều môi trường khác nhau.
- + Hỗ trợ lập trình nhanh: Python giúp rút ngắn thời gian phát triển phần mềm nhờ cú pháp linh hoạt và khả năng xử lý hiệu quả. Lập trình viên có thể tập trung vào việc xây dựng thuật toán thay vì mất nhiều thời gian cho các thao tác kỹ thuật phức tạp.
- + Cộng đồng hỗ trợ lớn: Python có cộng đồng người dùng rộng lớn trên toàn thế giới cùng với nhiều tài liệu, diễn đàn và khóa học hỗ trợ. Điều này giúp việc học tập, nghiên cứu và xử lý lỗi trở nên dễ dàng hơn.
- Lý do chọn Python để cài đặt
- + Dễ dàng triển khai thuật toán: Python có cú pháp đơn giản và trực quan, giúp việc xây dựng và mô phỏng thuật toán ElGamal trở nên dễ dàng hơn. Các đoạn mã được viết ngắn gọn nhưng vẫn đảm bảo tính rõ ràng và dễ hiểu.
- + Hỗ trợ mạnh mẽ cho tính toán: Python cung cấp nhiều thư viện hỗ trợ tính toán số lớn và thực hiện các phép toán học phức tạp. Đây là yếu tố quan trọng đối với các thuật toán mật mã yêu cầu độ chính xác cao.
- + Tiết kiệm thời gian phát triển: So với nhiều ngôn ngữ lập trình khác, Python giúp giảm số lượng dòng lệnh cần viết, từ đó tiết kiệm thời gian lập trình và kiểm thử chương trình.
- + Thuận tiện cho học tập và nghiên cứu: Python thường được sử dụng trong giảng dạy và nghiên cứu các lĩnh vực liên quan đến bảo mật và mật mã học nhờ cú pháp dễ hiểu và khả năng mô phỏng tốt.
- + Tích hợp với các công cụ phát triển hiện đại: Python tương thích với nhiều môi trường phát triển như PyCharm, Visual Studio Code và Jupyter Notebook, hỗ trợ hiệu quả cho việc lập trình, kiểm thử và gỡ lỗi chương trình.

- + Khả năng mở rộng cao: Python cho phép dễ dàng nâng cấp và tích hợp thêm các chức năng mới trong tương lai, phù hợp cho việc phát triển các ứng dụng liên quan đến bảo mật và mã hóa dữ liệu.

2.3.2.2 Ngôn ngữ C++

- Tổng quan về C++: C++ là một ngôn ngữ lập trình bậc trung (intermediate-level), được phát triển bởi Bjarne Stroustrup tại Bell Labs vào năm 1979 như một phần mở rộng của ngôn ngữ C. C++ kết hợp giữa sức mạnh của ngôn ngữ bậc thấp (cho phép thao tác trực tiếp với bộ nhớ) và tính năng của ngôn ngữ bậc cao. Ngôn ngữ này hỗ trợ đa mô hình lập trình như lập trình hướng đối tượng, lập trình thủ tục và lập trình tổng quát (generic programming). C++ được ứng dụng rộng rãi trong các hệ thống đòi hỏi hiệu năng cực cao như hệ điều hành, trình duyệt web, công cụ đồ họa và đặc biệt là các hệ thống bảo mật, mật mã học. Với khả năng tối ưu hóa tính toán, C++ là lựa chọn hàng đầu để cài đặt các hệ giải mã phức tạp như ElGamal trên các thiết bị có tài nguyên hạn chế hoặc yêu cầu tốc độ xử lý nhanh.
- Các đặc điểm nổi bật của C++
- + Hiệu năng vượt trội và kiểm soát tài nguyên: Là một ngôn ngữ biên dịch (compiled language), C++ cho phép chương trình chạy với tốc độ gần như tối đa của phần cứng. Khả năng quản lý bộ nhớ thủ công giúp lập trình viên tối ưu hóa việc sử dụng RAM và CPU, điều này cực kỳ quan trọng trong các phép toán số lớn của thuật toán ElGamal.
- + Hướng đối tượng mạnh mẽ: C++ hỗ trợ đầy đủ các tính chất của lập trình hướng đối tượng (OOP) như tính đóng gói, kế thừa, đa hình và trừu tượng. Điều này tương đồng với Python, giúp tổ chức mã nguồn một cách khoa học, dễ dàng tái sử dụng và quản lý các thành phần trong hệ thống mã hóa.
- + Thư viện mẫu chuẩn (STL): C++ sở hữu thư viện STL cực kỳ mạnh mẽ, cung cấp các cấu trúc dữ liệu và thuật toán tối ưu (như vector, map, set).

Ngoài ra, hệ sinh thái C++ có các thư viện mật mã chuyên dụng như OpenSSL hoặc Crypto++ hỗ trợ đặc lực cho việc xử lý số học và bảo mật.

- + Khả năng tương thích và đa nền tảng: C++ tuân theo các tiêu chuẩn ISO, giúp mã nguồn có thể biên dịch và chạy trên nhiều hệ điều hành từ Windows, Linux đến các hệ thống nhúng mà không cần thay đổi quá nhiều.
- + Khả năng can thiệp hệ thống: C++ cho phép làm việc với các bit, byte và địa chỉ bộ nhớ một cách chính xác, giúp việc triển khai các lớp vật lý của bảo mật thông tin trở nên hiệu quả hơn bất kỳ ngôn ngữ bậc cao nào khác.
- Lý do chọn C++ để cài đặt
- + Tốc độ xử lý toán học cực nhanh: Thuật toán ElGamal yêu cầu các phép tính lũy thừa rời rạc trên số nguyên lớn. C++ có khả năng thực thi các phép toán này với tốc độ nhanh hơn đáng kể so với các ngôn ngữ thông dịch, giúp giảm độ trễ khi mã hóa và giải mã dữ liệu thực tế.
- + Sự ổn định và bảo mật cao: Với khả năng kiểm soát chặt chẽ cách dữ liệu được lưu trữ và truyền tải trong bộ nhớ, C++ giúp lập trình viên hạn chế được các lỗi tràn bộ đệm (buffer overflow) nếu được thiết kế tốt, đảm bảo tính toàn vẹn cho hệ thống mật mã.
- + Hỗ trợ các thư viện số lớn chuyên dụng: Để giải quyết bài toán số nguyên lớn trong ElGamal, C++ có thể tích hợp dễ dàng với các thư viện như GMP (GNU Multiple Precision Arithmetic Library), cho phép xử lý các số có hàng nghìn bit với độ chính xác và hiệu suất tối đa.
- + Tiêu chuẩn trong công nghiệp bảo mật: C++ là ngôn ngữ tiêu chuẩn được sử dụng để xây dựng các lõi (core) bảo mật của hầu hết các phần mềm diệt virus, tường lửa và hệ thống mã hóa giao dịch ngân hàng hiện nay.
- + Khả năng tích hợp mạnh mẽ: C++ có thể dễ dàng được nhúng vào các ứng dụng khác hoặc làm việc trực tiếp với phần cứng, cho phép mở rộng

hệ thống mã hóa ElGamal lên các nền tảng thiết bị chuyên dụng hoặc hệ thống mạng phức tạp.

2.3.3 Cài đặt thuật toán, giao diện chương trình

2.3.3.1 Cài đặt thuật toán giao diện chương trình theo ngôn ngữ Python

- Các hàm liên quan
- + Hàm kiểm tra số nguyên tố

```
def E_kiemTraNguyenTo(so_kt: int) -> bool:
    if so_kt <= 1: return False
    if so_kt <= 3: return True
    if so_kt % 2 == 0 or so_kt % 3 == 0: return False
    for i in range(5, int(math.sqrt(so_kt)) + 1, 6):
        if so_kt % i == 0 or so_kt % (i + 2) == 0:
            return False
    return True
```

- + Hàm nguyên tố cùng nhau

```
def nguyenToCungNhau(ai: int, bi: int) -> bool:
    while bi != 0:
        ai, bi = bi, ai % bi
    return ai == 1
```


+ Hàm Euclid mở rộng

```
def euclidMoRong(a: int, m: int) -> int: 2+ usages
    m0 = m
    y = 0
    x = 1
    if m == 1:
        return 0
    while a > 1:
        q = a // m
        t = m
        m = a % m
        a = t
        t = y
        y = x - q * y
        x = t
    if x < 0:
        x = x + m0
    return x
```

+ Hàm tính lũy thừa Modulo $a^b \bmod n$

```
def E_LuyThuaModulo_(a: int, b: int, n: int) -> int:
    f = 1
    a = a % n
    while b > 0:
        if b % 2 == 1:
            f = (f * a) % n
        b = b // 2
        a = (a * a) % n
    return f
```

- Mã hóa

```
def E_MaHoa(du_lieu_vao: str, k_input: int) -> str: 1 usage
    global EsoA, EsoP, EsoD, txt_So_C1, txt_So_C2
    if du_lieu_vao.strip().isdigit() and int(du_lieu_vao) < EsoP:
        M_so = int(du_lieu_vao)
        C1 = E_LuyThuaModulo_(EsoA, k_input, EsoP)
        C2 = (M_so * E_LuyThuaModulo_(EsoD, k_input, EsoP)) % EsoP
        txt_So_C1 = str(C1)
        txt_So_C2 = str(C2)
        return f"S0:{C1},{C2}"
    else:
        mhE_temp1 = du_lieu_vao.encode('utf-16-le')
        base64_str = base64.b64encode(mhE_temp1).decode('utf-8')
        mh_temp2 = [ord(char) for char in base64_str]
        C1_co_dinh = E_LuyThuaModulo_(EsoA, k_input, EsoP)
        beta_k = E_LuyThuaModulo_(EsoD, k_input, EsoP)
        C2 = [0] * len(mh_temp2)
        so_c1 = ""
        so_c2 = ""

        for i in range(len(mh_temp2)):
            C2[i] = (mh_temp2[i] * beta_k) % EsoP
            so_c1 += str(C1_co_dinh) + " "
            so_c2 += str(C2[i]) + " "
        txt_So_C1 = so_c1.strip()
        txt_So_C2 = so_c2.strip()
        str_builder = [f"{C1_co_dinh},{C2[i]};" for i in range(len(C2))]
        full_pair_str = "".join(str_builder)
        return "CHUOI:" + base64.b64encode(full_pair_str.encode('utf-16-le')).decode('utf-8')
```

- Giải mã

```
def E_GiaiMa(chuoi_ma_hoa: str) -> str: 1 usage
    global EsoP, EsoX
    if chuoi_ma_hoa.startswith("S0:"):
        ban_ma_sach = chuoi_ma_hoa.replace(old: "S0:", new: "")
        pair = ban_ma_sach.split(',')
        C1 = int(pair[0])
        C2 = int(pair[1])

        # Tính toán chuẩn lý thuyết dùng Euclid mở rộng
        mau_so = E_LuyThuaModulo_(C1, EsoX, EsoP)
        s = euclidMoRong(mau_so, EsoP)

        M_giai_ma = (C2 * s) % EsoP
        return f"[Dạng Số] {M_giai_ma}"

    elif chuoi_ma_hoa.startswith("CHUOI:"):
        ban_ma_sach = chuoi_ma_hoa.replace(old: "CHUOI:", new: "")
        E_data1 = base64.b64decode(ban_ma_sach.encode('utf-8'))
        Egm_giaima = E_data1.decode('utf-16-le')
        pairs = [p for p in Egm_giaima.split(';') if p]
        length = len(pairs)
        C1 = [0] * length
        C2 = [0] * length
        for i in range(length):
            pair = pairs[i].split(',')
            C1[i] = int(pair[0])
            C2[i] = int(pair[1])
        M = [0] * length
        for i in range(length):
            # Áp dụng tương tự cho giải mã chuỗi ký tự
            mau_so = E_LuyThuaModulo_(C1[i], EsoX, EsoP)
            s = euclidMoRong(mau_so, EsoP)
            M[i] = (C2[i] * s) % EsoP

        str_builder = [chr(M[i]) for i in range(length)]
        base64_original = "".join(str_builder)
        van_ban_goc = base64.b64decode(base64_original.encode('utf-8')).decode('utf-16-le')
        return f"[Dạng Chuỗi] {van_ban_goc}"
    return "[LỖI] Không nhận diện được định dạng bản mã!"
```

- Hàm main

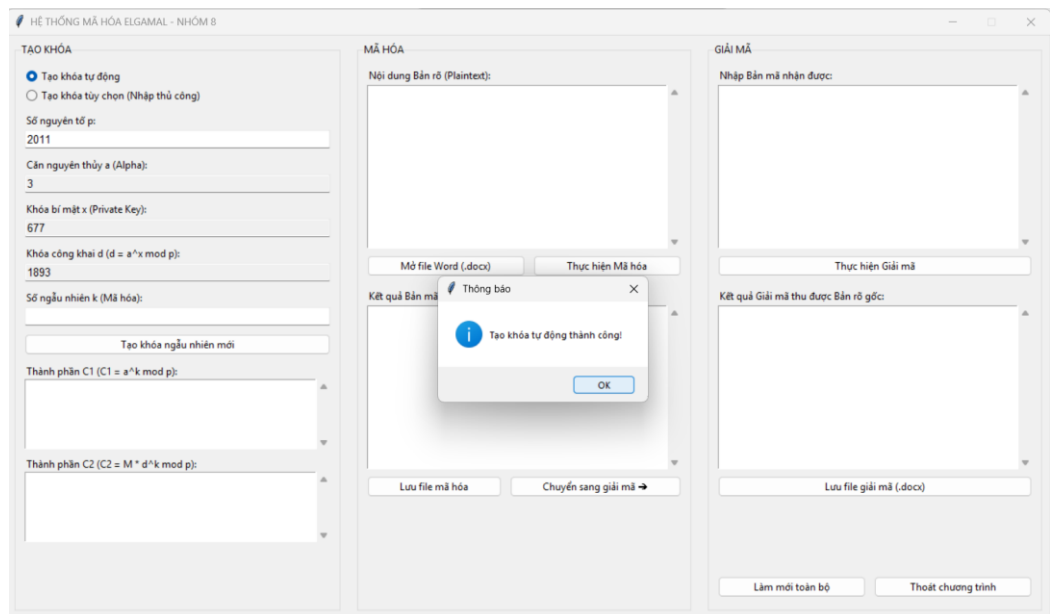
```
if __name__ == "__main__":
    print("=====")
    print("    HỆ THỐNG MẬT MÃ ELGAMAL    ")
    print("=====")
    while True:
        EsoP = int(input("1. Nhập số nguyên tố P : "))
        if E_kiemTraNguyenTo(EsoP):
            break
        else:
            print(f"[LỖI] Số {EsoP} không phải là số nguyên tố! Vui lòng nhập lại.\n")
    while True:
        EsoA = int(input("2. Nhập căn nguyên thủy alpha A : "))
        if EsoA >= EsoP:
            print(f"[LỖI] Phan tử sinh alpha A phải nhỏ hơn P ({EsoP})! Vui lòng nhập lại.\n")
        elif not nguyenToCungNhau(EsoA, EsoP):
            print(f"[LỖI] Alpha A ({EsoA}) và P ({EsoP}) phải nguyên tố cùng nhau! Vui lòng nhập lại.\n")
        else:
            break
```

```
EsoX = int(input("3. Nhập khóa bí mật X : "))
EsoD = E_LuyThuaModulo_(EsoA, EsoX, EsoP)
print("\n-----")
print(f" * KHÓA CÔNG KHAI : D = (P = {EsoP}, alpha = {EsoA}, YA = {EsoD})")
print(f" * KHÓA BÍ MẬT : X = {EsoX}")
print("-----\n")
du_lieu_nhap = input("-> Nhập bản rõ: ")
EsoK = int(input("-> Nhập số K để tính toán : "))
ban_ma_packet = E_MaHoa(du_lieu_nhap, EsoK)
print("\n--- KẾT QUẢ MÃ HÓA TỰ ĐỘNG ---")
print(f"Số k đã dùng: {EsoK}")
print(f"Kết quả sau khi mã hóa: {ban_ma_packet}")
print(f"Mảng số C1: {txt_So_C1}")
print(f"Mảng số C2: {txt_So_C2}")
ket_qua_giai_ma = E_GiaiMa(ban_ma_packet)
print("\n--- KẾT QUẢ GIẢI MÃ TỰ ĐỘNG ---")
print(f"Kết quả sau khi giải mã: {ket_qua_giai_ma}")
print("=====|=====")
```

- Giao diện chương trình

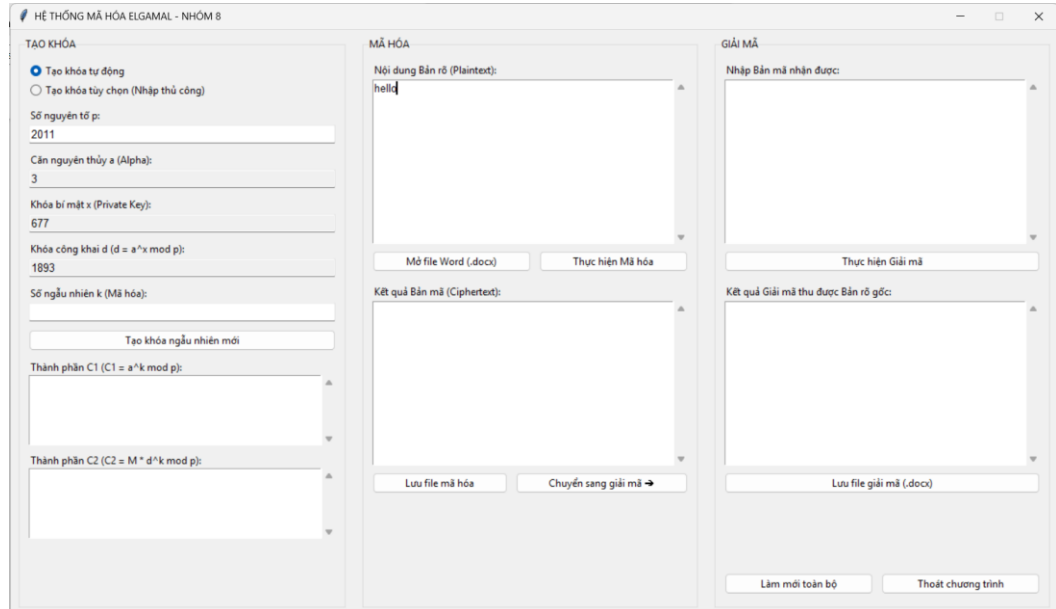
Bước 1: Tạo Khóa (Sinh các khóa a, x, d)

1. Nhập số nguyên tố lớn p (ví dụ 2011).
2. Chọn chế độ 'Tạo khóa tự động'.
3. Bấm vào nút 'Tạo khóa ngẫu nhiên mới'. Chương trình sẽ tự động quét tìm căn nguyên thủy a (Alpha), sinh khóa bí mật x và tính toán ra khóa công khai $d = a^x \bmod p$ như trên hình bên dưới:



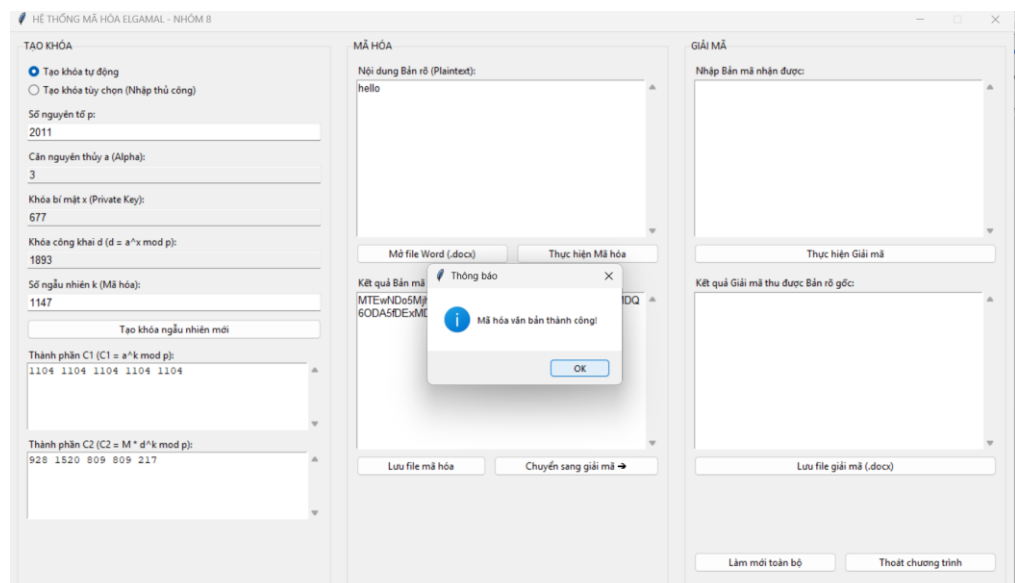
Bước 2: Nhập văn bản bản rõ cần mã hóa

1. Tại khung Mã hóa (cột giữa), người dùng nhập nội dung bản rõ cần mã hóa vào ô 'Nội dung Bản rõ (Plaintext)'.

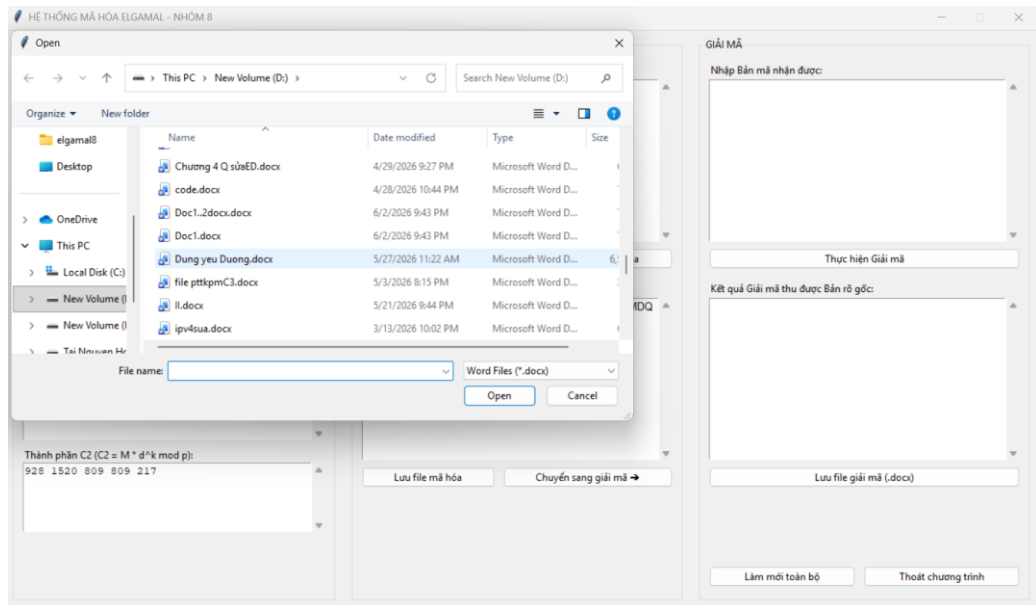


Bước 3: Thực hiện mã hóa bản rõ

1. Bấm vào nút 'Thực hiện Mã hóa'
2. Hệ thống sẽ tự động sinh số ngẫu nhiên k hiển thị ở cột bên trái.
3. Đồng thời, hệ thống tính toán giá trị C1, C2 cho từng ký tự/byte và cập nhật vào ô 'Thành phần C1' và 'Thành phần C2' ở cột bên trái.
4. Kết quả chuỗi bản mã dạng Base64 được hiển thị ở ô 'Kết quả Bản mã (Ciphertext)' ở cột giữa như sau:

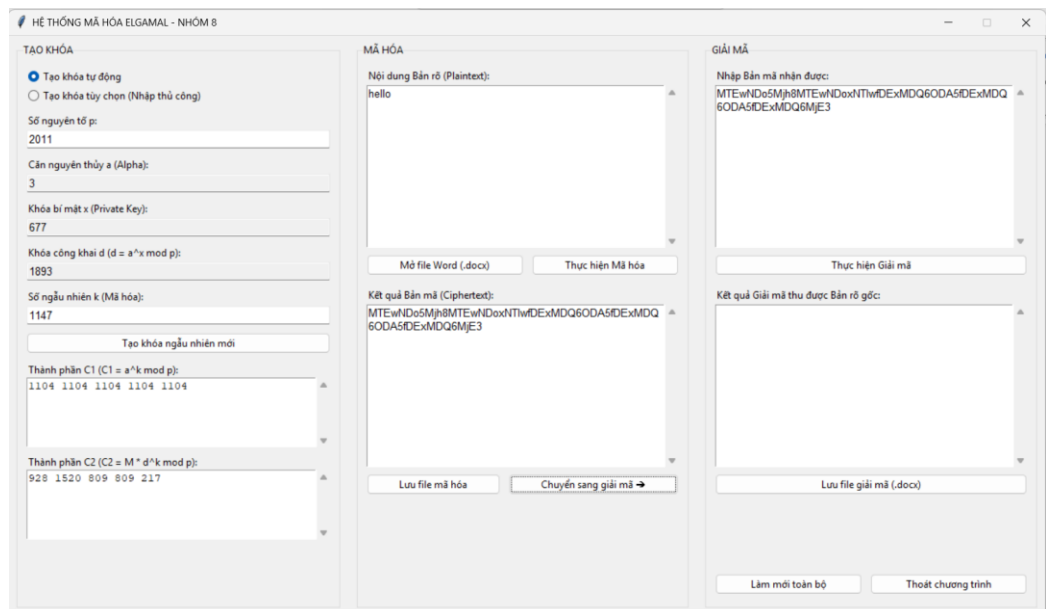


- + Trong trường hợp người dùng không muốn nhập bản rõ bằng tay có thể lấy bản rõ trong các file word trên thư mục của máy tính để mã hóa



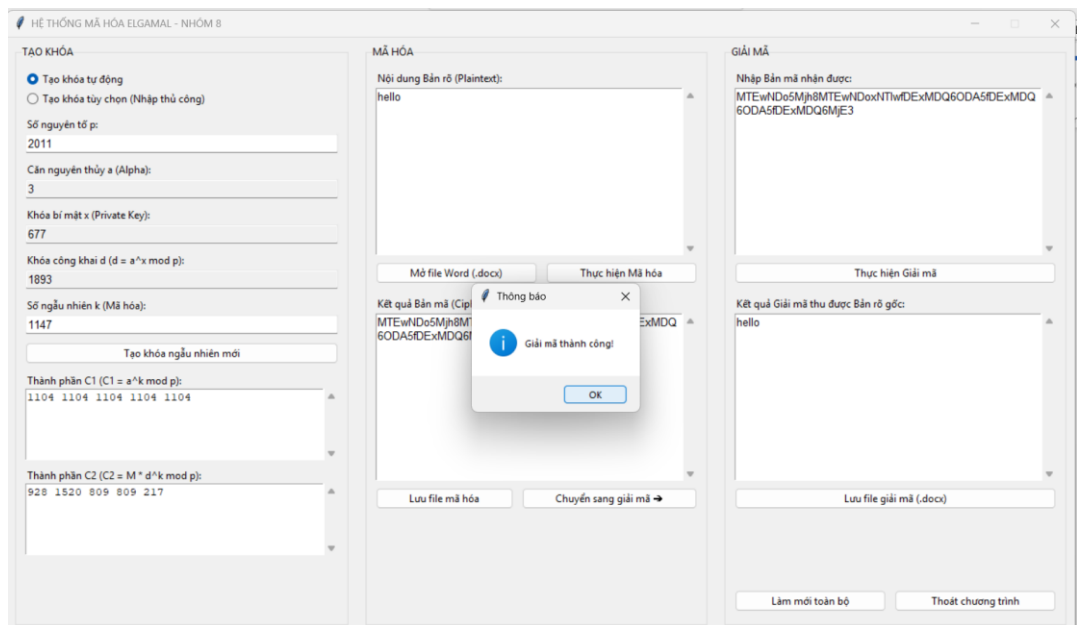
Bước 4: Chuyển bản mã sang phân hệ Giải mã

1. Bấm vào nút 'Chuyển sang giải mã →' ở cuối cột giữa.
2. Bản mã dạng Base64 từ ô 'Kết quả Bản mã' sẽ được sao chép và dán trực tiếp vào ô 'Nhập Bản mã nhận được' ở cột bên phải:

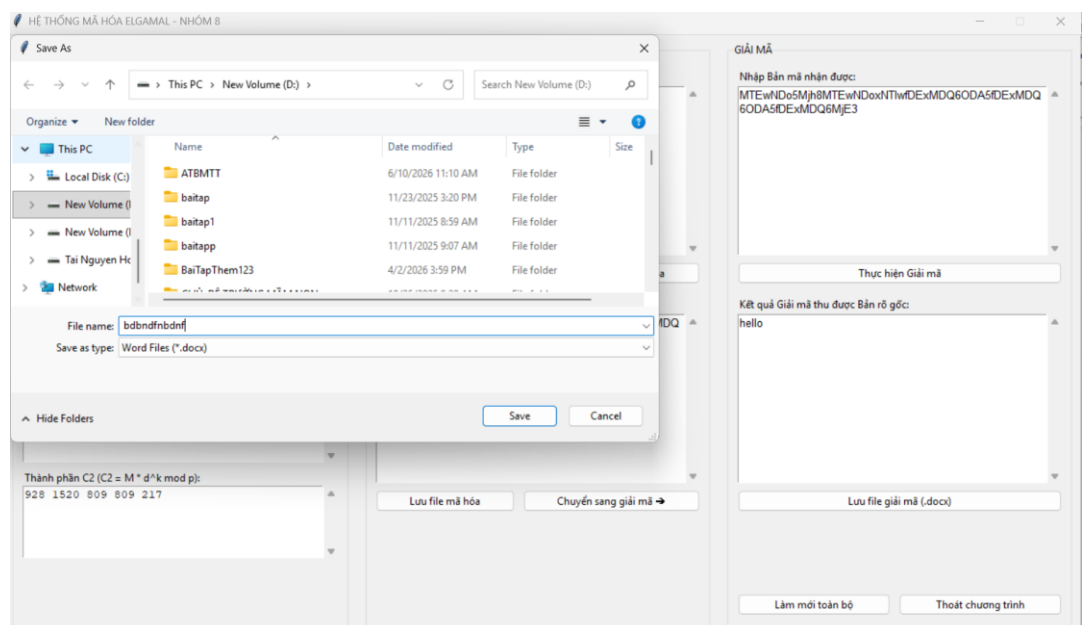


Bước 5: Thực hiện giải mã bản mã

1. Bấm vào nút 'Thực hiện Giải mã' ở cột bên phải.
2. Chương trình sẽ tự động giải mã chuỗi Base64 và tách các cặp số C1, C2, dùng khóa bí mật x để khử nhân tử ngẫu nhiên và khôi phục lại văn bản gốc chính xác.
3. Kết quả văn bản gốc hiển thị tại ô 'Kết quả Giải mã thu được Bản rõ gốc':



+ Người dùng có thể lưu lại file giải mã để thuận tiện có các lần thực hiện tiếp theo



2.3.3.2 Cài đặt thuật toán giao diện chương trình theo ngôn ngữ C++

- Các hàm liên quan
- + Hàm tính kiểm tra nguyên tố và lũy thừa Modulo $a^b \bmod n$

```
class ElGamal {
public:
    bool isPrime(long long n) {
        if (n <= 1) return false;
        if (n <= 3) return true;
        if (n % 2 == 0 || n % 3 == 0) return false;
        for (long long i = 5; i * i <= n; i += 6) {
            if (n % i == 0 || n % (i + 2) == 0) return false;
        }
        return true;
    }
    long long modPow(long long base, long long exp, long long mod) {
        long long result = 1;
        base %= mod;
        while (exp > 0) {
            if (exp % 2 == 1)
                result = (result * base) % mod;
            base = (base * base) % mod;
            exp /= 2;
        }
        return result;
    }
}
```

- + Thuật toán Euclid mở rộng tìm ước chung lớn nhất

```
long long extendedGCD(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long x1, y1;
    long long gcd = extendedGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return gcd;
}
```

- + Nghịch đảo modulo của a

```
long long modInverse(long long a, long long mod) {
    long long x, y;
    long long gcd = extendedGCD(a, mod, x, y);
    if (gcd != 1) return -1;
    return (x % mod + mod) % mod;
}
```

- Quá trình mã hóa bản rõ sang chuỗi ký tự

```
string base64Encode(const vector<unsigned char>& data) {
    static const string base64_chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
    string ret;
    int i = 0, j = 0;
    unsigned char char_array_3[3], char_array_4[4];

    for (size_t k = 0; k < data.size(); k++) {
        char_array_3[i++] = data[k];
        if (i == 3) {
            char_array_4[0] = (char_array_3[0] & 0xfc) >> 2;
            char_array_4[1] = ((char_array_3[0] & 0x03) << 4) + ((char_array_3[1] & 0xf0) >> 4);
            char_array_4[2] = ((char_array_3[1] & 0x0f) << 2) + ((char_array_3[2] & 0xc0) >> 6);
            char_array_4[3] = char_array_3[2] & 0x3f;
            for(i = 0; i < 4; i++) ret += base64_chars[char_array_4[i]];
            i = 0;
        }
    }
    if (i) {
        for(j = i; j < 3; j++) char_array_3[j] = '\0';
        char_array_4[0] = (char_array_3[0] & 0xfc) >> 2;
        char_array_4[1] = ((char_array_3[0] & 0x03) << 4) + ((char_array_3[1] & 0xf0) >> 4);
        char_array_4[2] = ((char_array_3[1] & 0x0f) << 2) + ((char_array_3[2] & 0xc0) >> 6);
        for (j = 0; (j < i + 1); j++) ret += base64_chars[char_array_4[j]];
        while((i++ < 3)) ret += '=';
    }
    return ret;
}
```

- Quá trình chuyển các chuỗi kỹ tự về bản rõ

```
vector<unsigned char> base64Decode(const string& encoded_string) {
    static const string base64_chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";
    int i = 0, j = 0;
    unsigned char char_array_4[4], char_array_3[3];
    vector<unsigned char> ret;

    for (size_t in_ = 0; in_ < encoded_string.size(); in_++) {
        char c = encoded_string[in_];
        if (c == '=') break;
        if (!isalnum(c) && c != '+' && c != '/') continue;

        char_array_4[i++] = c;
        if (i == 4) {
            for (i = 0; i < 4; i++) char_array_4[i] = base64_chars.find(char_array_4[i]);
            char_array_3[0] = (char_array_4[0] << 2) + ((char_array_4[1] & 0x30) >> 4);
            char_array_3[1] = ((char_array_4[1] & 0xf) << 4) + ((char_array_4[2] & 0x3c) >> 2);
            char_array_3[2] = ((char_array_4[2] & 0x3) << 6) + char_array_4[3];
            for (i = 0; i < 3; i++) ret.push_back(char_array_3[i]);
            i = 0;
        }
    }
    if (i) {
        for (j = i; j < 4; j++) char_array_4[j] = 0;
        for (j = 0; j < 4; j++) char_array_4[j] = base64_chars.find(char_array_4[j]);
        char_array_3[0] = (char_array_4[0] << 2) + ((char_array_4[1] & 0x30) >> 4);
        char_array_3[1] = ((char_array_4[1] & 0xf) << 4) + ((char_array_4[2] & 0x3c) >> 2);
        char_array_3[2] = ((char_array_4[2] & 0x3) << 6) + char_array_4[3];
        for (j = 0; j < i - 1; j++) ret.push_back(char_array_3[j]);
    }
    return ret;
}
```

- Mã hóa

```
string elgamalEncrypt(const string& banRo, long long q, long long a, long long ya, long long k, long long &outC1, long long &outC2) {
    ElGamal elgamal;
    vector<unsigned char> danhSachC1C2;

    for (size_t idx = 0; idx < banRo.size(); idx++) {
        long long m = static_cast<unsigned char>(banRo[idx]);
        long long c1 = elgamal.modPow(a, k, q);
        long long c2 = (m * elgamal.modPow(ya, k, q)) % q;

        if (idx == banRo.size() - 1) {
            outC1 = c1;
            outC2 = c2;
        }

        danhSachC1C2.push_back((c1 >> 24) & 0xFF);
        danhSachC1C2.push_back((c1 >> 16) & 0xFF);
        danhSachC1C2.push_back((c1 >> 8) & 0xFF);
        danhSachC1C2.push_back(c1 & 0xFF);

        danhSachC1C2.push_back((c2 >> 24) & 0xFF);
        danhSachC1C2.push_back((c2 >> 16) & 0xFF);
        danhSachC1C2.push_back((c2 >> 8) & 0xFF);
        danhSachC1C2.push_back(c2 & 0xFF);
    }

    if (banRo.empty()) {
        outC1 = elgamal.modPow(a, k, q);
        outC2 = 0;
    }

    return base64Encode(danhSachC1C2);
}
```

- Giải mã

```
string elgamalDecrypt(const string& banMaBase64, long long q, long long xa) {
    ElGamal elgamal;
    vector<unsigned char> danhSachC1C2 = base64Decode(banMaBase64);
    string ketQuaBanRo = "";

    for (size_t i = 0; i < danhSachC1C2.size(); i += 8) {
        if (i + 7 >= danhSachC1C2.size()) break;

        long long c1 = 0;
        c1 |= static_cast<long long>(danhSachC1C2[i]) << 24;
        c1 |= static_cast<long long>(danhSachC1C2[i+1]) << 16;
        c1 |= static_cast<long long>(danhSachC1C2[i+2]) << 8;
        c1 |= static_cast<long long>(danhSachC1C2[i+3]);

        long long b_val = 0;
        b_val |= static_cast<long long>(danhSachC1C2[i+4]) << 24;
        b_val |= static_cast<long long>(danhSachC1C2[i+5]) << 16;
        b_val |= static_cast<long long>(danhSachC1C2[i+6]) << 8;
        b_val |= static_cast<long long>(danhSachC1C2[i+7]);

        long long s = elgamal.modPow(c1, xa, q);
        long long s_inv = elgamal.modInverse(s, q);

        if (s_inv == -1) continue;

        long long m = (b_val * s_inv) % q;
        if (m < 0) m = (m + q) % q;

        ketQuaBanRo += static_cast<char>(m);
    }

    return ketQuaBanRo;
}
```

- Hàm main

```
int main() {
#ifdef _WIN32
system("chcp 65001 > nul");
#endif

ElGamal elgamal;
long long q, a, xa, k;
do {
    cout << "Nhap so nguyen to Q: ";
    cin >> q;
    if (!elgamal.isPrime(q)) {
        cout << "Q chua phai so nguyen to, moi ban nhap lai." << endl << endl;
    }
} while (!elgamal.isPrime(q));
cout << "Nhap can nguyen thuy a: ";
cin >> a;
cout << "Nhap khoa bi mat Xa: ";
cin >> xa;
cout << "Nhap khoa ngau nhien k: ";
cin >> k;

string dummy;
getline(cin, dummy);

long long ya = elgamal.modPow(a, xa, q);

cout << "\n=== HE THONG KHOA DA TINH TOAN ===" << endl;
cout << "-> Gia tri Ya tinh duoc: " << ya << endl;
cout << "-> Khoa cong khai Kpub: (" << q << ", " << a << ", " << ya << ")" << endl;
cout << "-> Khoa bi mat Kpr: (" << q << ", " << xa << ")" << endl;
cout << "===== " << endl << endl;

string banRoNhap;
cout << "Nhap ban ro bat ky (Moi ky tu tren ban phim): ";
getline(cin, banRoNhap);

long long hienC1 = 0, hienC2 = 0;
string banMaBase64 = elgamal.encrypt(banRoNhap, q, a, ya, k, hienC1, hienC2);

cout << "\n-> Ket qua C1: " << hienC1 << endl;
cout << "-> Ket qua C2: " << hienC2 << endl;
cout << "-> BAN MA THU DUOC (Chuoi chu Base64):\n" << banMaBase64 << endl << endl;

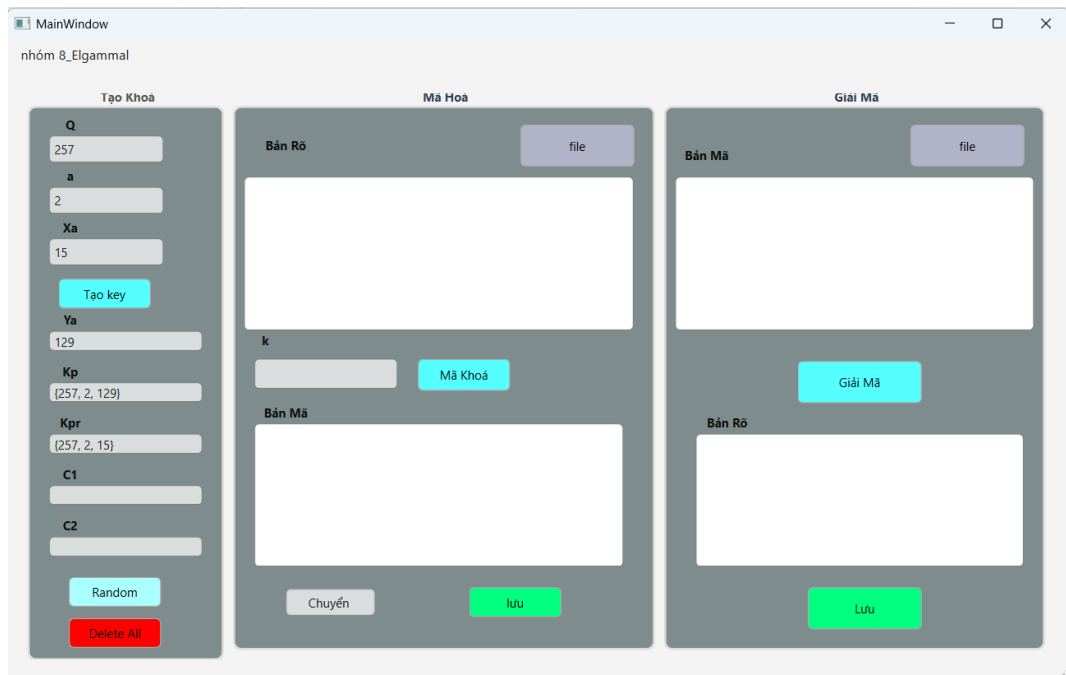
string banRoGiaiMa = elgamal.decrypt(banMaBase64, q, xa);
cout << "-> KET QUA SAU KHI GIAI MA:\n" << banRoGiaiMa << endl;

return 0;
}
```

- Giao diện chương trình

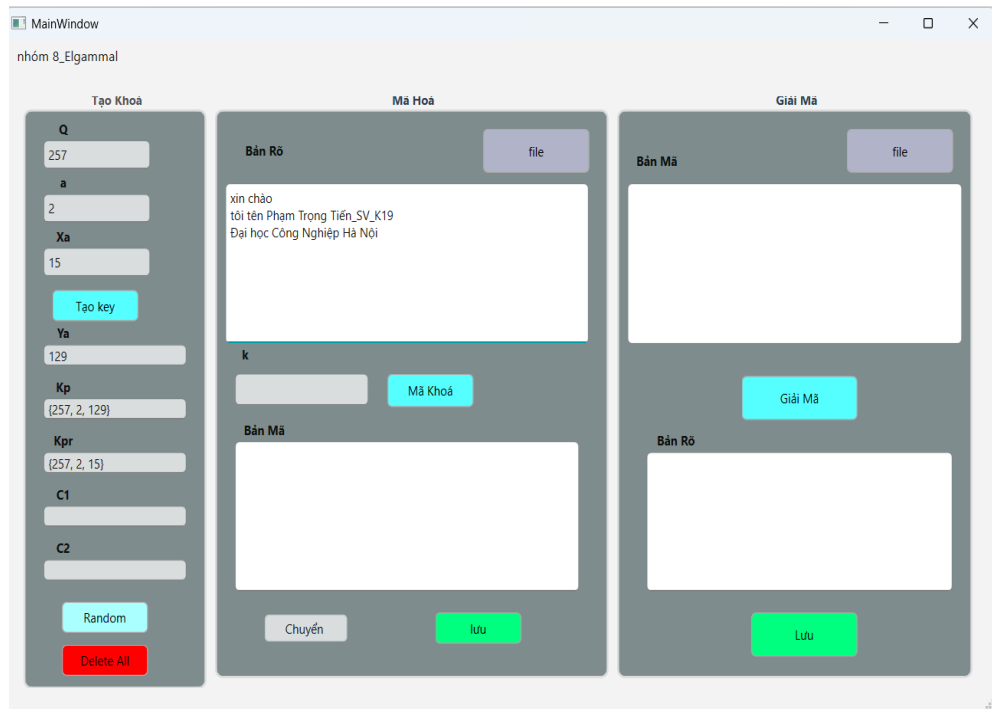
Bước 1: Tạo Khóa (Sinh các khóa a , X_a , Y_a)

1. Nhập số nguyên tố lớn Q (ví dụ 257).
2. Chọn số căn nguyên thủy a
3. Chọn X_a
4. Bấm “Tạo Key”



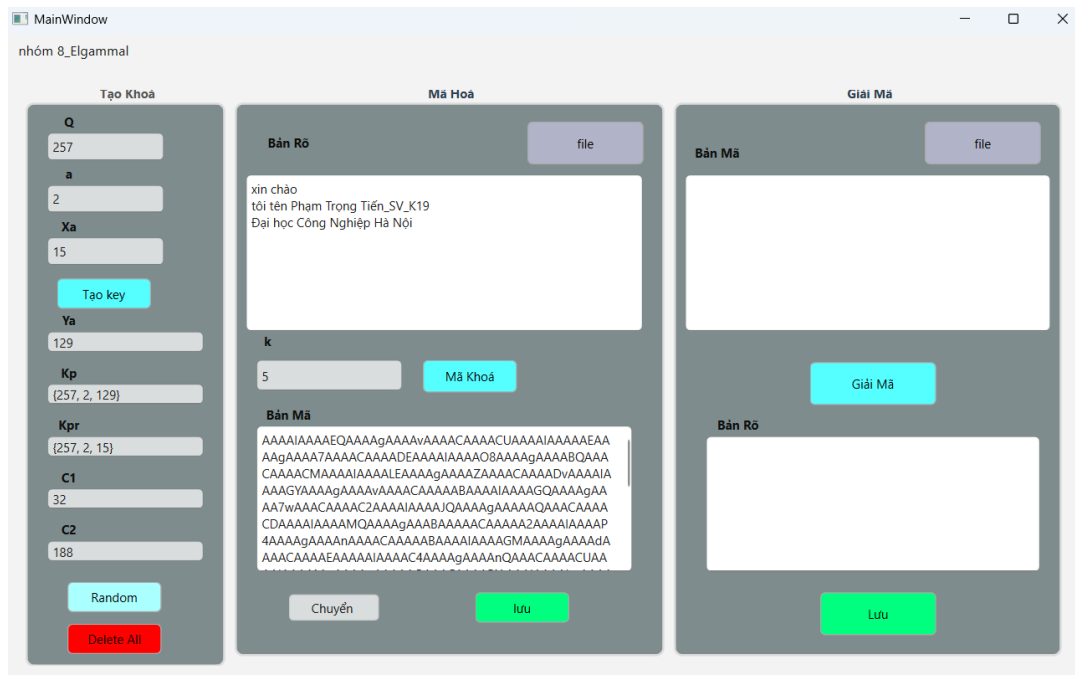
Bước 2: Nhập văn bản bản rõ cần mã hóa

Tại khung Mã hóa (cột giữa), người dùng nhập nội dung bản rõ cần mã hóa vào ô 'Bản rõ (Plaintext)'



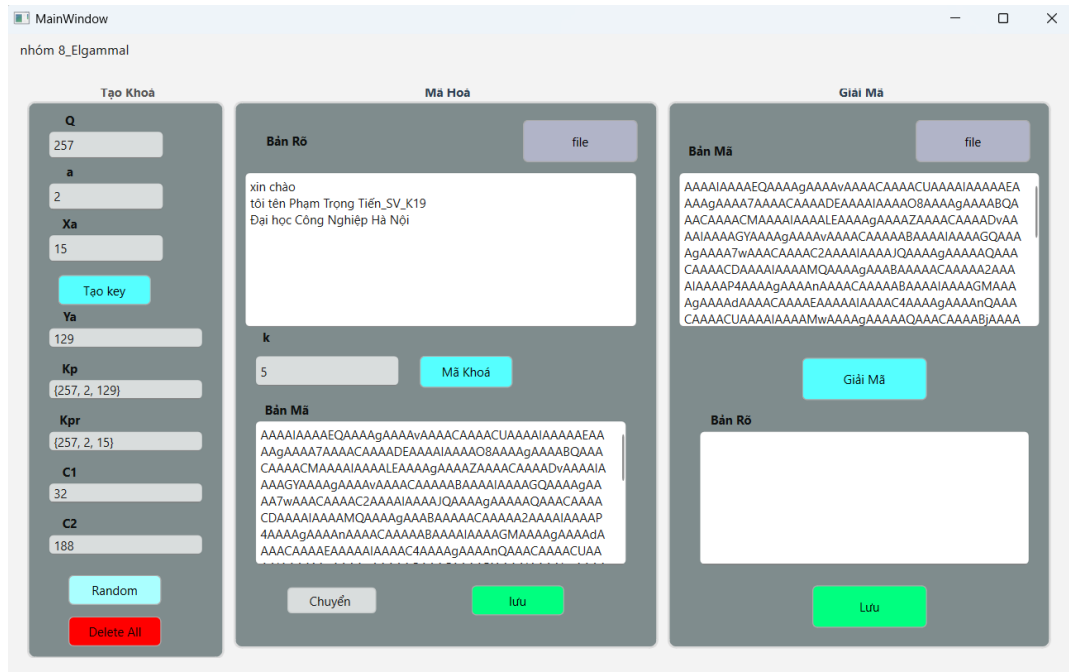
Bước 3: Thực hiện mã hóa bản rõ

1. Chọn số ngẫu nhiên k
2. Bấm vào nút 'Thực hiện Mã hóa'
3. Hệ thống tính toán giá trị $C1$, $C2$ cho 1 ký tự/byte và cập nhật vào ô ' $C1$ ' và ' $C2$ ' ở cột bên trái.
4. Kết quả chuỗi bản mã dạng Base64 được hiển thị ở ô 'Kết quả Bản mã (Ciphertext)' ở cột giữa như sau:



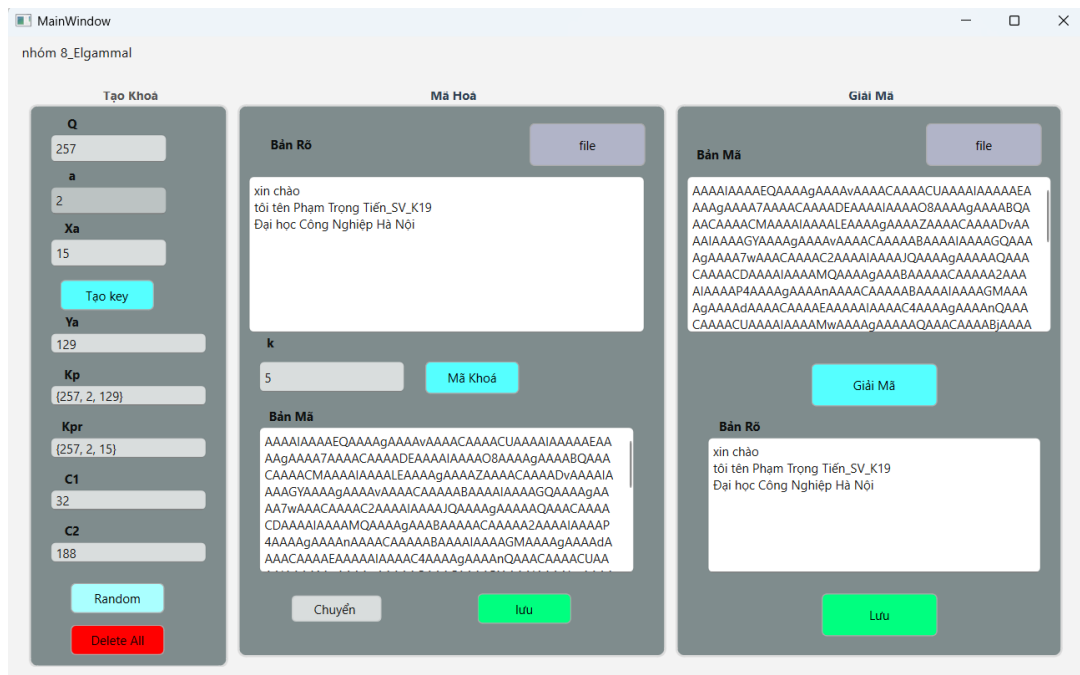
Bước 4: Chuyển bản mã sang phân hệ Giải mã

1. Bấm vào nút 'Chuyển sang giải mã →' ở cuối cột giữa.
2. Bản mã dạng Base64 từ ô 'Bản mã' sẽ được sao chép và dán trực tiếp vào ô 'Bản mã' ở cột bên phải:



Bước 5: Thực hiện giải mã bản mã

1. Bấm vào nút 'Thực hiện Giải mã' ở cột bên phải.
2. Chương trình sẽ tự động giải mã chuỗi Base64 và tách các cặp số C1, C2, dùng khóa bí mật x để khử nhân tử ngẫu nhiên và khôi phục lại văn bản gốc chính xác.
3. Kết quả văn bản gốc hiển thị tại ô 'Bản rõ':



CHƯƠNG 3. KẾT LUẬN VÀ BÀI HỌC KINH NGHIỆM

3.1 Kiến thức và kỹ năng đã học được

Thông qua quá trình nghiên cứu và xây dựng chương trình mã hóa/giải mã ElGamal, nhóm đã tích lũy được những kiến thức và kỹ năng quan trọng sau:

3.1.1 Kiến thức chuyên môn

Hệ mật mã khóa công khai: Hiểu rõ cơ sở lý thuyết và nguyên lý hoạt động của hệ mật mã bất đối xứng, cụ thể là hệ mật ElGamal dựa trên bài toán logarit rời rạc.

Các thuật toán nền tảng: Lĩnh hội và áp dụng thành thạo các thuật toán toán học hỗ trợ cho mật mã học như:

- Thuật toán Euclid và Euclid mở rộng để tìm ước chung lớn nhất và nghịch đảo modulo.
- Thuật toán bình phương và nhân để tính lũy thừa modulo nhanh ($x^n \bmod m$).
- Định lý Fermat nhỏ, hàm phi Euler và cách kiểm tra số nguyên tố.
- Thuật toán Baby-Step Giant-Step để tìm phần tử sinh.

An toàn thông tin: Nhận thức được vai trò của các đặc tính bảo mật như tính riêng tư, tính toàn vẹn, khả dụng và tính chống chối bỏ trong truyền tin. Đánh giá được ưu, nhược điểm và độ an toàn của hệ mã ElGamal so với các hệ mật khác như RSA.

3.1.2 Kỹ năng thực hành và lập trình

Lập trình đa ngôn ngữ: Phát triển kỹ năng cài đặt thuật toán trên nhiều ngôn ngữ lập trình khác nhau như Python và C++.

Xây dựng ứng dụng thực tế: Kỹ năng thiết kế giao diện người dùng (GUI) cho các chương trình ứng dụng web và desktop.

Xử lý dữ liệu: Học được kinh nghiệm quan trọng trong việc xử lý file, đặc biệt là cách lấy bản rõ từ các định dạng file phức tạp như .docx và xử lý dữ liệu đầu vào của người dùng để đảm bảo chương trình hoạt động mượt mà.

3.1.3 Kỹ năng mềm và bài học kinh nghiệm

Kỹ năng nghiên cứu tài liệu: Cải thiện khả năng đọc hiểu tài liệu chuyên ngành bằng tiếng Anh, vì đây là nguồn tài liệu chính thống và phong phú nhất về mật mã học.

Làm việc nhóm: Rèn luyện kỹ năng tổ chức hoạt động nhóm, phân chia công việc hiệu quả và phối hợp giữa các thành viên để hoàn thành dự án đúng tiến độ.

Tư duy giải quyết vấn đề: Học cách đối mặt và vượt qua các khó khăn kỹ thuật trong quá trình triển khai thực tế, từ đó rút ra những bài học kinh nghiệm quý báu cho các dự án tương lai.

3.2 Bài học kinh nghiệm

Trong quá trình nghiên cứu và xây dựng chương trình mã hóa và giải mã thuật toán ElGamal sử dụng ngôn ngữ Python và C++, nhóm chúng em đã rút ra một số bài học kinh nghiệm quan trọng như sau:

- Hiểu vững lý thuyết là nền tảng cốt lõi: Thuật toán ElGamal không chỉ đơn thuần là một công thức, mà còn gắn liền với các khái niệm toán học như số nguyên tố lớn, logarit rời rạc, số nguyên tố Sophie Germain, ... Việc hiểu rõ lý thuyết đã giúp nhóm dễ dàng hơn trong việc chuyển hóa sang mã nguồn thực tế.
- Mỗi ngôn ngữ có thế mạnh và thách thức riêng: **Python** hỗ trợ xử lý số nguyên lớn (big integer) cùng hệ sinh thái thư viện mã hóa phong phú, giúp việc cài đặt và thử nghiệm các thuật toán mật mã trở nên nhanh chóng, ngắn gọn và dễ dàng hơn. Trong khi đó, **C++** cho phép kiểm soát chi tiết tài nguyên hệ thống, tốc độ thực thi cao và khả năng tối ưu hóa tốt, phù hợp với các ứng dụng yêu cầu hiệu năng lớn. Tuy nhiên, việc quản lý kiểu dữ liệu, bộ nhớ và xử lý nhập xuất trong C++ thường phức tạp hơn Python. Qua quá trình thực hiện, nhóm nhận thấy việc lựa chọn ngôn ngữ lập trình cần dựa trên yêu cầu cụ thể của bài toán, cân bằng giữa tốc độ phát triển, hiệu năng và khả năng mở rộng của hệ thống.

- Thử nghiệm và kiểm thử là bắt buộc trong bảo mật: Trong mật mã học, chỉ một lỗi nhỏ cũng có thể khiến toàn bộ quá trình mã hóa hoặc giải mã thất bại. Vì vậy, nhóm đã đặc biệt chú trọng đến việc kiểm thử từng hàm, từng bước xử lý và so sánh kết quả giữa hai ngôn ngữ để đảm bảo tính chính xác.
- Làm việc nhóm và phân chia công việc hiệu quả: Với đề tài có tính chất kỹ thuật cao và gồm nhiều phần như lý thuyết, triển khai, giao diện và báo cáo, nhóm đã học được cách lập kế hoạch phân công rõ ràng, hỗ trợ lẫn nhau và cập nhật tiến độ định kỳ. Điều này giúp quá trình phát triển diễn ra trơn tru hơn.
- Tính ứng dụng và ý nghĩa thực tiễn tạo động lực học tập: Khi nhận ra thuật toán ElGamal đang được ứng dụng trong các hệ thống bảo mật thực tế như GPG/PGP, nhóm cảm thấy việc tìm hiểu trở nên thú vị hơn. Qua đó, các thành viên hiểu rằng kiến thức lý thuyết trong trường học có thể thực sự áp dụng vào thế giới thực.

3.3 Đề xuất về tính khả thi của chủ đề nghiên cứu, những thuận lợi, khó khăn

Tính khả thi:

- Elgamal là một trong những hệ mã hóa công khai phổ biến và quan trọng, được sử dụng rộng rãi trong thực tế
- Việc xây dựng chương trình mã hóa và giải mã Elgamal sẽ giúp sinh viên nắm vững kiến thức về nguyên lý hoạt động, cách triển khai của hệ thống mã hóa này.
- Đây là một chủ đề phù hợp với kiến thức mã hóa, lý thuyết số và lập trình mà sinh viên đã học trong quá trình học tập.

Thuận lợi:

- Có nhiều tài liệu, ví dụ mã nguồn mẫu có sẵn trên Internet để tham khảo.

- Các bước thực hiện mã hóa và giải mã Elgamal được mô tả rõ ràng trong nhiều tài liệu, dễ hiểu và triển khai.
- Có thể tham khảo nhiều nguồn tham khảo khác nhau để hoàn thiện báo cáo.
- Khi hoàn thành, sẽ có một chương trình hoàn chỉnh và hiểu rõ các khái niệm liên quan.

Khó khăn:

- Có các kiến thức nền tảng về lý thuyết số, đại số, mã hóa để hiểu rõ nguyên lý hoạt động của Elgamal.
- Cần kỹ năng lập trình để xây dựng chương trình mã hóa và giải mã, có thể phức tạp hơn so với một số chủ đề khác.
- Cần thêm các tính năng nâng cao, ví dụ như giao diện người dùng, sẽ tăng thêm độ phức tạp của dự án.
- Cần quan tâm đến các vấn đề như hiệu suất, tính bảo mật, xử lý ngoại lệ,...

TÀI LIỆU THAM KHẢO

- [1] Nguyễn Xuân Dũng, Bảo mật thông tin - Mô hình và ứng dụng, NXB Thống Kê, 2009.
- [2] Bùi Doãn Khanh, Nguyễn Đình Thúc, Mã hóa thông tin - Lý thuyết và ứng dụng, NXB Lao động xã hội, 2011.
- [3] William Stallings, Cryptography and Network Security Principles and Practices, Fourth Edition, Prentice Hall, 2005.